

Università degli Studi dell'Aquila

Dipartimento di Ingegneria e Scienze dell'Informazione e Matematica

Corso di Laurea in Informatica

Tesi di Laurea

Modellazione e rilevamento di eventi complessi in sistemi di videosorveglianza multimodale

Relatore:
Prof. Fabio Persia

Candidato:
Roberto Ringoli
Matricola 291717

Anno Accademico 2025/2026

Abstract

Un sistema di videosorveglianza registra più di quanto una persona possa effettivamente guardare; allo stesso tempo, in ambito forense ciò che conta maggiormente è il rilevamento degli eventi complessi, consistenti in relazioni temporali tra fatti semplici, come una persona che corre e poco dopo sale su un veicolo. WATCHOUT, un sistema di videosorveglianza in via di sviluppo in collaborazione con Hitachi, rileva eventi complessi in video in modalità multimodale: modelli visione-linguaggio e audio trasformano il video in intervalli annotati, salvati in un database relazionale, e ogni evento è una query SQL costruita con gli operatori temporali del linguaggio ISEQL, quindi verificabile passo per passo. All'inizio di questo lavoro, però, gli eventi definiti erano stati cablati nel codice, e aggiungerne uno richiedeva di modificare e riavviare il sistema. L'ISEQL Event Model Builder, di una tesi precedente, permetteva di disegnare un evento su una linea temporale, ma non di definirlo formalmente.

Pertanto questa tesi integra i due sistemi. Un ponte tra i due modelli dei dati riconduce gli intervalli eterogenei di WATCHOUT al modello del Builder, ricavando la posizione di ogni partecipante dal suo tipo. Un compilatore traduce il modello di un evento in SQL usando per ogni operatore la stessa definizione degli eventi predefiniti, in modo che il sistema abbia una semantica consistente. Un editor nel browser permette di costruire l'evento disegnandolo su una linea temporale o scrivendolo in ISEQL. Il passo successivo consisterà nell'integrare il sistema con un'implementazione efficiente degli operatori ISEQL, già pubblicata in un prestigioso lavoro di ricerca.

Il risultato è che un evento disegnato e salvato viene rilevato in aggiunta a quelli predefiniti, senza scrivere codice SQL né modificare o riavviare il sistema, e resta verificabile perché la query che lo rileva è leggibile. Una prima prova con utenti non esperti, affiancata da una valutazione euristica, giudica il sistema efficace ma non estremamente pratico: per chi non conosce la notazione, l'editor grafico è l'unica strada percorribile. Per cui si dovrebbe migliorare ulteriormente l'interfaccia, e non il modello degli eventi né il compilatore.

Indice

Abstract **i**

1 Introduzione **1**

2 Stato dell'arte **4**
2.1 La videosorveglianza come archivio non indicizzato 4
2.2 Dal fotogramma all'evento: i livelli di astrazione 6
2.3 Rilevazione di eventi complessi: definire o apprendere 9
2.3.1 Approcci dichiarativi: l'evento come query 9
2.3.2 Approcci basati su apprendimento: l'evento come classe 10
2.4 I modelli visione-linguaggio nel livello percettivo 11
2.5 Il punto di partenza: WATCHOUT 13

3 Query a intervalli per la modellazione di eventi **15**
3.1 Intervalli e relazioni temporali 15
3.1.1 L'algebra di Allen 15
3.1.2 Perché le tredici relazioni non bastano 16
3.2 ISEQL 17
3.2.1 Il modello dei dati 17
3.2.2 Gli otto operatori temporali 18
3.2.3 Selezione, proiezione e operazioni insiemistiche 19
3.2.4 ISEQL+: strettezza e robustezza 20
3.2.5 Dall'algebra al DBMS 21
3.2.6 Dall'algebra al SQL 22
3.3 Modellare eventi senza scrivere query 22
3.3.1 L'ISEQL Event Model Builder 23
3.3.2 Cosa il Builder non faceva 24
3.4 Gli eventi di WATCHOUT in ISEQL 24

4 Tecnologie utilizzate **27**
4.1 Base di dati: SQLite 28
4.2 Backend: Python e FastAPI 29
4.3 Frontend: Svelte, SvelteKit e TypeScript 31
4.4 Livello percettivo: modelli e librerie 32

5 Architettura del sistema **33**
5.1 Vista d'insieme . . . 33
5.2 Il ponte tra i due modelli dei dati . . . 35
5.2.1 Ricavare la posizione dal tipo . . . 35
5.2.2 Lo scoping per analisi . . . 36
5.3 Il modello di evento . . . 37
5.4 Il compilatore . . . 38
5.4.1 Da modello a SQL . . . 38
5.4.2 Una sola definizione per operatore . . . 40
5.4.3 Cosa il compilatore rifiuta . . . 41
5.4.4 Le soglie . . . 41
5.5 Registro e servizio . . . 42
5.5.1 Il salvataggio nel registro . . . 42
5.5.2 La rilevazione . . . 43
5.5.3 Il ciclo, dimostrato . . . 43
5.6 L'editor . . . 43
5.6.1 La linea temporale . . . 44
5.6.2 Operatori e soglie . . . 46
5.6.3 Salvare l'evento . . . 48
5.7 Sviluppo in parallelo . . . 48

6 Funzionamento ed espressività **49**
6.1 Un evento dall'editor al rilevamento . . . 49
6.2 Espressività . . . 51
6.2.1 I sedici eventi come modelli . . . 51
6.2.2 Cosa il sistema aggiunge . . . 51
6.2.3 Cosa non si può esprimere . . . 52
6.3 Il repository video . . . 53
6.4 Ciò che l'uso ha fatto emergere . . . 54
6.5 Soddisfazione degli utenti . . . 55
6.5.1 La prova con gli utenti . . . 55
6.5.2 Valutazione euristica dell'interfaccia . . . 58
6.5.3 Cosa significa . . . 58

7 Conclusioni **61**
7.1 Cosa è stato fatto . . . 61
7.2 Limiti . . . 62
7.3 Sviluppi futuri . . . 63

Bibliografia **65**

Elenco delle figure

2.1 Un fotogramma di una scena di sorveglianza con le entità che il livello percettivo deve riconoscere: persone, un veicolo, un oggetto trasportato. I riquadri sono le rilevazioni del livello percettivo: in rosso le persone, in giallo l'oggetto, in blu i veicoli. 5

2.2 I tre livelli di astrazione: dalle osservazioni per fotogramma (Λ), agli intervalli (M), agli eventi complessi (C), definiti come espressioni su M. Figura ripresa dal paper VIS MODE [8]. 8

2.3 Un evento rilevato e i fotogrammi di evidenza che lo sostengono: ogni rilevazione si può ripercorrere fino alle osservazioni che l'hanno prodotta. Schermata di WATCHOUT; la stessa sequenza compare nel paper VIS MODE [8]. 12

2.4 L'interfaccia di analisi di WATCHOUT: a sinistra le analisi precedenti, al centro il caricamento del video e la scelta della condizione, a destra i risultati, qui un evento *fight* tra due persone. 14

4.1 Le parti del sistema e i loro collegamenti. Il frontend parla con il backend via HTTP; il backend scrive gli intervalli prodotti dai modelli nel database SQLite e vi esegue le query generate dal compilatore. . . 27

5.1 L'architettura del sistema integrato: l'editor produce un modello, il salvataggio lo scrive nel registro, la rilevazione lo compila in SQL ed esegue la query sugli intervalli prodotti dal livello percettivo. 34

5.2 L'editor con un evento multimodale: l'insieme `s1` contiene i due intervalli sonori (*engine*, *tire_squeal*) legati da SP, l'insieme `s2` i due visivi (*running*, *enter_or_exit_vehicle*) legati da Bef, e l'espressione insiemistica in basso a destra li unisce. I distintivi circolari tra le barre mostrano l'operatore di ciascuna coppia. 44

5.3 La finestra dell'operatore, aperta dal distintivo tra due barre: la coppia a cui si riferisce, l'operatore scelto fra gli otto, e un controllo per ciascun parametro che quell'operatore prende. Qui SP, che prende δ e non ε, con δ senza limite. 47

5.4 La finestra di un insieme: il nome, gli intervalli che contiene con le loro posizioni argomentali, la proiezione come lista ordinata di colonne, e le condizioni incrociate. 47

6.1 Il disegno dell'evento: due intervalli sulla linea temporale e il distintivo *Bef* tra i due. 50

Capitolo 1

Introduzione

Il problema: due sistemi complementari

Un impianto di videosorveglianza produce ogni giorno più ore di registrazione di quante una persona possa guardarne. Il valore di quelle registrazioni non sta nei singoli fotogrammi, ma negli **eventi** che contengono: un veicolo che si allontana in fretta subito dopo uno sparo, un oggetto che passa da una persona a un'altra, qualcuno che sosta troppo a lungo davanti a un ingresso. Sono **eventi complessi** nel senso che nessuno di essi è visibile in un fotogramma isolato. Ognuno è una *relazione temporale* tra fatti più semplici: una persona vicina a un veicolo per almeno dieci secondi, uno sparo seguito entro pochi secondi da una fuga.

La ricerca sulla rilevazione di eventi complessi (*Complex Event Detection*, CED) ha proposto da tempo di separare i due livelli. Un **livello percettivo** riconosce oggetti, persone e suoni e li annota con il loro intervallo di tempo; un **livello logico** definisce gli eventi come query su quegli intervalli. In ambito forense questa separazione ha un vantaggio decisivo: se l'evento è il risultato di una query, la sua rilevazione si può *verificare passo per passo*, cosa che un classificatore addestrato end-to-end non permette. **ISEQL**, l'*Interval-based Surveillance Event Query Language* [1], è un linguaggio nato con questo scopo. Si tratta di un'algebra relazionale su intervalli, i cui operatori temporali derivano dalle relazioni di Allen [2] e sono parametrizzati da soglie.

WATCHOUT, il sistema su cui si fonda questa tesi, applica questo schema alla sorveglianza multimodale. Modelli visione-linguaggio e modelli audio producono intervalli annotati, che vengono salvati in un database relazionale, e ogni evento è una query SQL. Il sistema funziona ed è stato valutato su un dataset di trenta scene. All'inizio di questo lavoro, però, i suoi eventi, sedici in tutto tra visivi, sonori e multimodali, erano *scritti a mano* nel codice dell'applicazione: per aggiungerne

uno bisognava scrivere SQL, modificare il backend e riavviarlo. Era un'operazione alla portata di chi aveva scritto il sistema, non di chi lo usa.

Esisteva già uno strumento per il problema opposto. L'**ISEQL Event Model Builder**, sviluppato in una tesi precedente, permette di disegnare intervalli su una linea temporale e ricava da solo l'espressione ISEQL corrispondente, riconoscendo quale operatore temporale lega ogni coppia di intervalli. È uno strumento di *modellazione*: produce la definizione di un evento, ma non è collegato a nessun dato e non esegue nulla.

I due sistemi sono quindi complementari. Uno può *esprimere* eventi ma non rilevarli, l'altro può *rilevarli* ma non esprimerne di nuovi. La domanda da cui nasce questa tesi è se sia possibile farne un sistema solo, in cui un evento viene disegnato, salvato, e da quel momento rilevato accanto a quelli predefiniti, senza che nessuno debba scrivere una riga di SQL.

Il contributo

Il lavoro descritto in questa tesi è l'integrazione dei due sistemi, a partire da una scelta su come eseguire le espressioni ISEQL. Per i suoi operatori esiste un'implementazione efficiente, basata su join a scansione del piano e pubblicata su rivista da Piatov et al. [3]; in questa tesi, invece, le espressioni sono tradotte in SQL ed eseguite dal database relazionale di WATCHOUT, e il passaggio a quell'implementazione è previsto come passo successivo (Sezione 3.2.5). Il primo ostacolo all'integrazione è che i due sistemi non parlano degli stessi dati. Il Builder ragiona su intervalli uniformi, con gli argomenti in posizioni fisse; WATCHOUT conserva dati eterogenei, relazioni visive tra oggetti, intervalli sonori e partecipanti di tipo diverso, senza un ordine tra loro. Il punto di partenza è quindi un *ponte tra i due modelli dei dati*, che ricava la posizione di ogni argomento dal suo tipo.

Su questo ponte poggia un *compilatore*, che traduce un modello di evento ISEQL in una query SQL eseguibile sul database di WATCHOUT: intervalli, operatori temporali con le loro soglie, condizioni tra partecipanti, proiezione ed espressioni insiemistiche. Per ogni operatore il compilatore non usa una definizione propria, ma la stessa già impiegata dagli eventi predefiniti, così che l'intero sistema abbia *una sola semantica*. Gli eventi vivono in un *registro* unico, dove un evento definito dall'utente è una riga come quelle dei sedici predefiniti e viene compilato dallo stesso percorso al momento della rilevazione: diventa un dato del sistema, non una parte del suo codice.

Per chi usa il sistema, tutto questo si presenta come un *editor* nel browser. Gli intervalli si disegnano su una linea temporale, la disposizione delle barre propone l'operatore temporale di ogni coppia e l'autore lo conferma o lo cambia; accanto, un editor testuale ISEQL lavora sullo stesso modello.

Resta da mostrare che la traduzione è corretta, e la *verifica* si basa sull'equivalenza. Ciascuno dei sedici eventi è stato riscritto come modello, e le occorrenze che produce una volta compilato devono coincidere con quelle della query scritta a mano corrispondente, sugli stessi dati registrati.

Il risultato è che un evento disegnato nell'editor e salvato compare nell'interfaccia di analisi e viene rilevato sugli stessi video degli altri, con lo stesso grado di verificabilità, perché la query che lo rileva resta visibile e leggibile.

Struttura della tesi

Il resto della tesi è organizzato come segue.

Capitolo 2: Stato dell'arte. Presenta il dominio: la gestione dei flussi video nella sorveglianza, la rilevazione di eventi complessi e la letteratura più recente, in cui i modelli visione-linguaggio hanno cambiato il livello percettivo.

Capitolo 3: Query a intervalli per la modellazione di eventi. È dedicato alla teoria su cui si fonda il lavoro, dalle relazioni di Allen a ISEQL e alle sue estensioni, fino agli strumenti esistenti per modellare eventi senza scrivere query.

Capitolo 4: Tecnologie utilizzate. Presenta gli strumenti impiegati a partire dal problema che ciascuno risolve e dalle scelte di progetto che ne sono seguite.

Capitolo 5: Architettura del sistema. Descrive il sistema realizzato e le decisioni di progetto che l'hanno determinato.

Capitolo 6: Funzionamento ed espressività. Mostra il sistema in uso: l'espressività delle query ottenibili, il repository di video su cui sono state provate, come ne è stata verificata la correttezza e cosa ne pensa un primo gruppo di utenti non esperti.

Capitolo 7: Conclusioni. Riassume quanto fatto e indica gli sviluppi futuri.

Capitolo 2

Stato dell'arte

Questo capitolo colloca la tesi nel suo contesto. Parte dalla videosorveglianza come pratica e come fonte di dati, passa per la distinzione tra percezione ed eventi che regge tutto il lavoro, confronta le due famiglie di approcci alla rilevazione di eventi complessi, e arriva alla letteratura più recente, in cui i modelli visione-linguaggio hanno trasformato il livello percettivo. Si chiude con WATCHOUT, il sistema da cui questa tesi parte, presentato come istanza di quello schema e con il limite che la tesi si propone di rimuovere.

2.1 La videosorveglianza come archivio non indicizzato

La **videosorveglianza** nasce come estensione dello sguardo di un operatore: una o più telecamere inquadrano luoghi che una persona non potrebbe presidiare di persona, e le immagini arrivano in una sala di controllo dove qualcuno le osserva. Per decenni il sistema è stato tutto qui, e la telecamera era un occhio in più. Con il passaggio al digitale e con il calo del costo di sensori e memoria, il numero di telecamere per impianto è cresciuto molto più del numero di persone incaricate di guardarle. Un operatore che segue decine di monitor non può seguirli davvero. Il risultato è che *la maggior parte del video registrato non viene mai guardata da nessuno.*

Questo cambia la natura del dato. Un impianto moderno registra in continuo, in genere per un periodo fissato da norme o da politiche interne, dopo il quale sovrascrive. In quel periodo l'archivio contiene ore di video per ogni telecamera, quasi tutte prive di interesse. Il valore dell'archivio si concentra in pochi istanti, e il problema pratico è trovarli. Si distinguono così due usi del sistema. Nell'**uso in tempo**

reale la domanda è se stia accadendo qualcosa adesso, e la risposta deve arrivare in tempo per intervenire. Nell'**uso forense** la domanda è cosa sia accaduto in un certo luogo in un certo intervallo, e la risposta deve essere accurata, documentata e ripercorribile, perché spesso è destinata a un'indagine o a un procedimento. Questa tesi si colloca nel secondo caso.

Nell'uso forense l'archivio video si comporta come *una base di dati che nessuno ha ancora indicizzato*. Chi cerca non è interessato ai fotogrammi, ma a fatti che si estendono nel tempo e coinvolgono più attori: una persona che si avvicina a un veicolo e vi resta accanto, un oggetto che cambia di mano, un veicolo che parte di scatto. Sono queste le unità naturali della ricerca, e infatti i benchmark del settore sono definiti in termini di eventi. **VIRAT** [4], uno dei più usati, raccoglie ore di riprese da telecamere fisse e aeree su parcheggi e aree pubbliche, annotate con eventi come una persona che sale su un veicolo o che vi carica un oggetto. L'evento è definito da chi lo cerca, ha un inizio e una fine, e si compone di elementi più semplici che il video mostra uno alla volta.

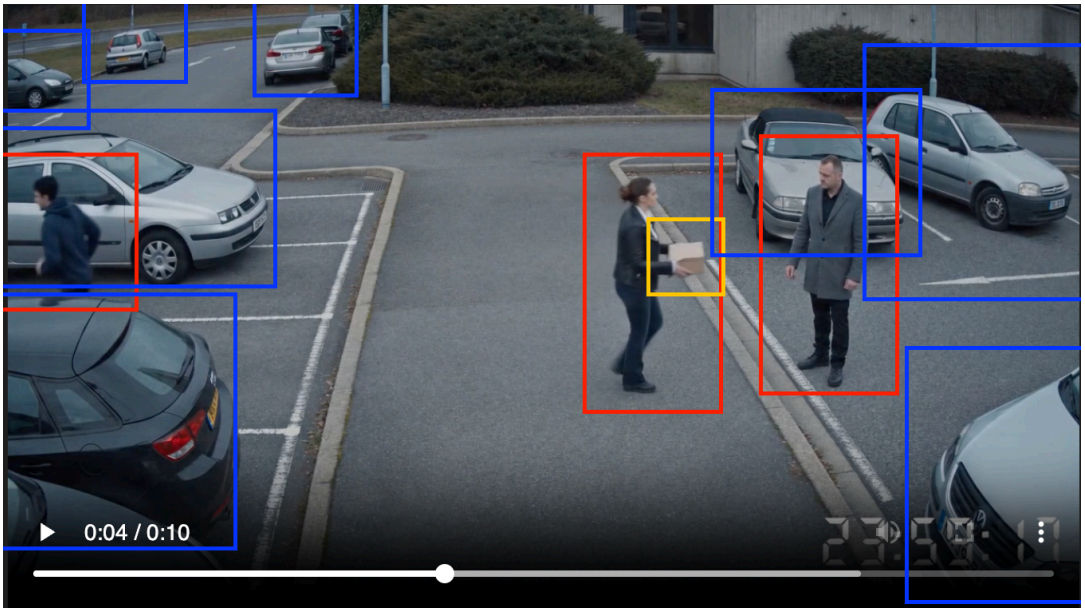


Figura 2.1: Un fotogramma di una scena di sorveglianza con le entità che il livello percettivo deve riconoscere: persone, un veicolo, un oggetto trasportato. I riquadri sono le rilevazioni del livello percettivo: in rosso le persone, in giallo l'oggetto, in blu i veicoli.

C'è infine una dimensione che la parola videosorveglianza nasconde. Molte telecamere registrano anche l'audio, e alcuni degli eventi più rilevanti in ambito forense sono udibili prima che visibili, o udibili soltanto: uno sparo, una frenata, un vetro infranto possono avvenire fuori dall'inquadratura. Un sistema che tratta il solo flusso video rinuncia a questa informazione. Nel seguito, quando si parla di sistemi

multimodali, si intende che il livello percettivo produce intervalli da entrambe le sorgenti, e che gli eventi possono essere definiti combinandole.

2.2 Dal fotogramma all'evento: i livelli di astrazione

Tra il video grezzo e l'evento che interessa a chi indaga ci sono più livelli di astrazione, e conviene distinguerli con cura perché è su questa distinzione che si regge l'intera tesi.

Al livello più basso ci sono i fotogrammi, cioè matrici di pixel. Il primo passo di qualunque sistema automatico è riconoscere in ciascun fotogramma le entità presenti: persone, veicoli, oggetti trasportabili, e per l'audio le classi di suono. È il **livello percettivo**, ed è quello su cui la visione artificiale ha fatto i progressi maggiori negli ultimi quindici anni, passando da pipeline costruite a mano, con sottrazione dello sfondo e descrittori locali, a rilevatori appresi end-to-end come **YOLO** [5], e più di recente a modelli visione-linguaggio in grado di descrivere una scena in linguaggio naturale. Per l'audio il percorso è stato analogo, con reti pre-addestrate su grandi tassonomie di suoni ambientali come **PANNs** [6]. Qualunque sia la tecnica, l'uscita di questo livello ha la stessa forma: un'etichetta, con la sua confidenza, riferita a un istante o a un breve intervallo.

Conviene dare a questo livello un nome e una forma. Chiamiamo **etichettatura puntuale** l'insieme Λ delle osservazioni prodotte dal livello percettivo: ciascuna è un predicato applicato ad alcuni attori e riferito a un singolo fotogramma,

$$\Lambda = \{\, (P_i,\ a_1, \ldots, a_{n_i},\ t_j) \,\}, \qquad i = 1, \ldots, k, \quad j = 1, \ldots, m, \tag{2.1}$$

dove l'indice i varia sui k predicati del vocabolario $P_1, \ldots, P_k$ (*porta un pacco*, *è vicino a un veicolo*, *sparo*), n_i è il numero di argomenti dell'i-esimo predicato, $a_1, \ldots, a_{n_i}$ sono gli argomenti del predicato, cioè identificatori di persone, oggetti o veicoli, e l'indice j varia sugli m fotogrammi campionati $t_1, \ldots, t_m$ del video. Un'osservazione come (*hasPkg*, p_1, x, 90) dice che al fotogramma 90 la persona p_1 porta l'oggetto x, e nulla di più.

Poiché n_i cambia da un predicato all'altro, le tuple di Λ non hanno tutte lo stesso numero di attributi: *corre* ha un solo argomento, *porta un pacco* ne ha due. Una relazione unica che le contenga tutte deve prevedere tante colonne per gli argomenti quante ne servono al predicato che ne ha di più, e lasciare vuote quelle in eccesso nelle altre tuple. È una rappresentazione **denormalizzata**: comoda da interrogare,

perché ogni osservazione è una sola riga con tutti i suoi partecipanti, ma con colonne vuote e con un numero massimo di argomenti fissato dallo schema. Se serve, la si può **normalizzare**, separando le osservazioni dai loro partecipanti in due relazioni, con una riga per osservazione e una riga per ciascun partecipante. Lo stesso vale per la relazione M introdotta qui sotto, e la forma normalizzata è quella che WATCHOUT adotta nel suo database, come si vedrà nella Sezione 3.3.

Il livello percettivo, però, lavora fotogramma per fotogramma, mentre le cose che accadono hanno una durata. Una persona non è "vicina a un veicolo" in un fotogramma: lo è per un tratto di tempo, con un inizio e una fine. Il passo successivo consiste quindi nel raggruppare le osservazioni consecutive che riguardano la stessa entità e la stessa relazione in un **intervallo**, annotato con chi vi partecipa. Persia et al. [7] chiamano questo il **livello intermedio**: predicati come "una persona entra in una zona", "una persona porta un pacco", "una persona è dentro un veicolo", ciascuno con il proprio intervallo di frame. A questo livello *il video è già diventato una tabella*: una riga per intervallo, con un predicato, i suoi argomenti e i frame di inizio e fine. È l'**etichettatura a intervalli** M,

$$M = \{\, r_1, \ldots, r_N \,\}, \qquad r_h = (P_i,\ a_1, \ldots, a_{n_i},\ T_s,\ T_e), \quad h = 1, \ldots, N, \tag{2.2}$$

dove l'indice h numera le N tuple di M, distinto dall'indice j dei fotogrammi nell'equazione (2.1). Ogni tupla r_h è fatta come un'osservazione di Λ, con una sola differenza: al posto del singolo fotogramma t_j ci sono i due estremi $T_s \leq T_e$ dell'intervallo di fotogrammi, estremi inclusi, durante il quale il predicato vale. Il predicato P_i e i suoi n_i argomenti sono invece quelli di Λ: il passaggio a M non introduce predicati nuovi.

Il passaggio da Λ a M consolida le osservazioni consecutive dello stesso predicato sugli stessi attori in un'unica tupla, tollerando brevi interruzioni dovute a occlusioni o a errori del modello. Se la persona p_1 porta l'oggetto x dal fotogramma 30 al fotogramma 90, le osservazioni $(\mathit{hasPkg}, p_1, x, 30), \ldots, (\mathit{hasPkg}, p_1, x, 90)$ di Λ diventano in M la sola tupla

$$r_h = (\mathit{hasPkg},\ p_1,\ x,\ 30,\ 90).$$

Per indicare un attributo di una tupla si usa la notazione puntata, come per i campi di un oggetto: in questo esempio il predicato è $r_h.P = \mathit{hasPkg}$, il primo argomento è $r_h.a_1 = p_1$, e l'intervallo è $[r_h.T_s,\ r_h.T_e] = [30, 90]$. È qui che nasce la struttura su cui ragionerà tutto il resto, ed è per questo che la Figura 2.2 la mette al centro.

È su questa tabella che si definiscono gli **eventi di alto livello**, quelli che un operatore cercherebbe. Un evento complesso non è un'osservazione in più, ma una

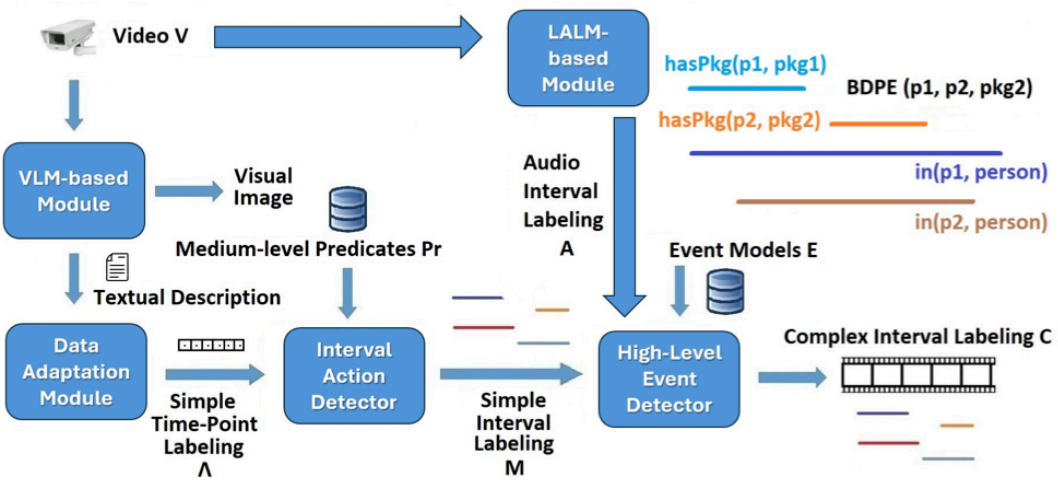


Figura 2.2: I tre livelli di astrazione: dalle osservazioni per fotogramma (Λ), agli intervalli (M), agli eventi complessi (C), definiti come espressioni su M. Figura ripresa dal paper VIS MODE [8].

relazione temporale tra intervalli: un passaggio di oggetto è un intervallo in cui una persona porta un pacco, seguito entro pochi secondi da un intervallo in cui lo porta un'altra; una fuga in veicolo è un intervallo in cui una persona entra in un veicolo, seguito da uno in cui il veicolo lascia la scena. Chiamiamo C l'insieme di questi eventi. Le relazioni possibili tra due intervalli sono poche e note da tempo, e sono l'oggetto del Capitolo 3. Ciò che conta qui è che l'evento complesso è definito *per composizione*, e che la composizione è *dichiarativa*: si può scrivere come una query su M.

Questa stratificazione ha una conseguenza che nell'uso forense diventa il requisito principale. Se un evento è il risultato di una query su intervalli, la sua rilevazione è **ripercorribile**: si può risalire agli intervalli che l'hanno prodotta, da lì ai fotogrammi che hanno generato quegli intervalli, e verificare ogni passaggio. Un classificatore addestrato a riconoscere direttamente "passaggio di oggetto" dal video non offre questa possibilità: risponde sì o no, e il motivo della risposta resta dentro i pesi della rete. Inoltre ogni nuovo evento richiederebbe nuovi dati etichettati e un nuovo addestramento, mentre nell'approccio dichiarativo richiede una nuova query sugli stessi dati. La separazione tra percezione ed eventi permette infine di sostituire il livello percettivo senza toccare le definizioni degli eventi, e ciò è esattamente quanto è avvenuto negli ultimi anni con l'arrivo dei modelli visione-linguaggio, come si vede nella Sezione 2.4.

2.3 Rilevazione di eventi complessi: definire o apprendere

Il problema di riconoscere eventi composti in un flusso video ha una storia lunga, e le soluzioni proposte si dividono in due famiglie che differiscono su un punto preciso: *dove sta la definizione dell'evento*. Nella prima famiglia la definizione è scritta da una persona, in un linguaggio formale, e il sistema la esegue. Nella seconda la definizione è appresa da esempi, e sta nei parametri di un modello. La distinzione conta perché determina chi può aggiungere un evento nuovo e quanto la rilevazione sia spiegabile.

2.3.1 Approcci dichiarativi: l'evento come query

I primi lavori sul riconoscimento di eventi complessi trattano l'evento come una **gerarchia di azioni semplici** legate da vincoli temporali: l'evento ha una definizione esplicita, e il ragionamento temporale è la parte centrale del sistema. La linea di ricerca a cui questa tesi appartiene direttamente è quella di Helmer e Persia, che hanno portato questa idea dentro il modello relazionale. Il lavoro presentato a ICSC 2016 [1], poi esteso e formalizzato su rivista [9], introduce **ISEQL**: un'algebra relazionale i cui operatori di join sono relazioni temporali tra intervalli, derivate da quelle di Allen [2] e parametrizzate da soglie che rendono il confronto tollerante al rumore. Un evento di alto livello diventa un'espressione dell'algebra, e la sua rilevazione una query eseguita su una tabella di intervalli.

Vale la pena vedere subito la forma di questi operatori, perché è la stessa che il sistema descritto nel Capitolo 5 compila in SQL. Prese due tuple r e s di M, la relazione *before* di Allen richiede che r finisca prima che s cominci; ISEQL vi aggiunge una soglia δ che limita la distanza tra i due intervalli:

$$r \ \mathrm{Bef}_{\delta} \ s \quad \Longleftrightarrow \quad r.T_e \leq s.T_s \ \wedge \ s.T_s - r.T_e \leq \delta. \tag{2.3}$$

Senza la soglia, uno sparo all'inizio del video e una fuga dieci minuti dopo sarebbero "prima" l'uno dell'altra esattamente come se fossero avvenuti a due secondi di distanza; con la soglia, la relazione esprime la vicinanza temporale che l'evento richiede. Ogni operatore di ISEQL è un join di questo tipo, e il Capitolo 3 li presenta tutti.

Con un operatore così definito, un evento è un'espressione di algebra relazionale. Il passaggio di un oggetto tra due persone, per esempio, è la composizione di due

intervalli *hasPkg* sullo stesso oggetto, con persone diverse, il secondo che segue il primo entro δ:

$$handoff \;=\; \pi_{p_1,\,p_2,\,x}\Big(\sigma_{\,M_1.p \neq M_2.p \,\wedge\, M_1.x = M_2.x}\big(M_1 \;\mathrm{Bef}_{\delta}\; M_2\big)\Big), \tag{2.4}$$

dove M_1 e M_2 sono due copie della relazione degli intervalli *hasPkg*. Tutto ciò che serve per definire l'evento sta in questa riga: quali intervalli, in che relazione temporale, con quali vincoli sui partecipanti. Ed è una riga che si può leggere, discutere e correggere. Gli autori valutano il linguaggio su tre dataset, tra cui le riprese di una stazione ferroviaria e di un parcheggio universitario, con precisione e richiamo quasi perfetti sulla maggior parte delle classi di evento e tempi di risposta di pochi millisecondi per un intero video.

Il passo successivo riguarda **chi definisce gli eventi**. Nel framework interattivo di Persia et al. [10] l'architettura a tre livelli, elaborazione delle immagini, rilevazione degli intervalli e rilevazione degli eventi di alto livello, è esposta attraverso un'interfaccia grafica che permette di ispezionare lo stato del sistema e di *creare definizioni di eventi durante l'uso*, senza fermare il sistema. Il lavoro sull'etichettatura dei frame con eventi a intervalli [7] fissa il livello intermedio, con predicati come una persona che entra in una zona o che porta un pacco. La recente estensione ISEQL+ [11] generalizza le soglie con operatori di strettezza e un parametro di robustezza e applica il linguaggio a un dominio diverso, quello sanitario, a conferma che l'approccio non dipende dalla sorveglianza. Sul lato dell'esecuzione, Piatov et al. [3] mostrano che i join su intervalli per le relazioni estese di Allen si possono valutare con algoritmi a scansione del piano efficienti nell'uso della cache, più veloci di diverse volte rispetto allo stato dell'arte precedente. Il Capitolo 3 riprende questi lavori nel dettaglio, perché ne costituiscono la base teorica.

Il pregio comune di questi approcci è la **trasparenza**: l'evento è una formula, il risultato è l'insieme delle tuple che la soddisfano, e ogni rilevazione si può ripercorrere. Il limite, altrettanto comune, è che scrivere quella formula richiede di conoscere il formalismo. Nella pratica le definizioni finiscono scritte da chi ha costruito il sistema, e gli strumenti per permettere a un operatore di definirle da solo sono rimasti pochi. È esattamente il punto su cui interviene questa tesi.

2.3.2 Approcci basati su apprendimento: l'evento come classe

La seconda famiglia rinuncia alla definizione esplicita. Un modello viene addestrato su video etichettati a riconoscere direttamente la classe di evento, oppure, nella

variante più diffusa in sorveglianza, a riconoscere ciò che è **anomalo** rispetto al comportamento normale della scena. La survey di Barbosa et al. [12] fa il punto sulla rilevazione di anomalie con supervisione debole e con più modalità, video e audio insieme, e mostra quanto il campo sia attivo.

Questi metodi hanno il vantaggio di non richiedere alcuna definizione formale e di adattarsi a eventi difficili da descrivere a parole. Hanno però due limiti che nell'uso forense pesano molto. Il primo è la **spiegabilità**: il modello risponde con un punteggio, e il motivo del punteggio resta nei suoi pesi; per un'indagine questo non basta. Il secondo è il **costo di un evento nuovo**: ogni classe richiede esempi etichettati e un nuovo addestramento, mentre nell'approccio dichiarativo richiede una nuova query sugli stessi dati. Per questo il lavoro qui presentato resta nella prima famiglia per quanto riguarda gli eventi, e affida all'apprendimento soltanto il livello percettivo, come discusso nella sezione seguente.

2.4 I modelli visione-linguaggio nel livello percettivo

Fino a pochi anni fa il livello percettivo di un sistema come quelli della Sezione 2.3 era costruito con rilevatori specializzati: un modello per le persone, uno per i veicoli, un tracker per seguirli, e per ogni nuovo predicato di medio livello un pezzo di codice scritto apposta. Il cambiamento più importante della letteratura recente è che questo strato può essere sostituito da un unico modello generalista a cui si chiede, in linguaggio naturale, cosa c'è nel fotogramma.

I **modelli visione-linguaggio** (VLM) accoppiano un codificatore visivo a un modello linguistico e producono testo a partire da un'immagine e da una domanda. La survey di Tang et al. [13] ricostruisce come questa famiglia si sia estesa dal fotogramma singolo al video, e mette in evidenza i limiti che restano aperti: il costo di elaborare sequenze lunghe, la finestra di contesto oltre la quale il modello perde ciò che ha visto all'inizio, e la difficoltà di collocare nel tempo con precisione ciò che descrive. Sono limiti che riguardano il ragionamento temporale, non la percezione. Un VLM descrive bene un fotogramma; è nel tenere insieme minuti di video che fatica.

Questo suggerisce una divisione dei compiti precisa, ed è quella adottata dal gruppo di ricerca in cui questa tesi si colloca. In **VIS MODE** [8] il VLM è usato come *osservatore*: descrive ogni fotogramma campionato con un vocabolario chiuso di predicati e identificatori persistenti degli attori, le osservazioni vengono consolidate in intervalli e la rilevazione degli eventi complessi è affidata a un motore ISEQL

esterno al modello. La percezione è appresa, il ragionamento è dichiarativo, e i due livelli comunicano attraverso una tabella di intervalli, cioè esattamente la struttura descritta nella Sezione 2.2. Che il ragionamento sia esterno al modello si vede anche dai risultati: l'evento non è "visto" dal VLM, che descrive solo fotogrammi; è ricostruito dagli intervalli, e per ogni rilevazione il sistema può mostrare i fotogrammi che la sostengono (Figura 2.3).

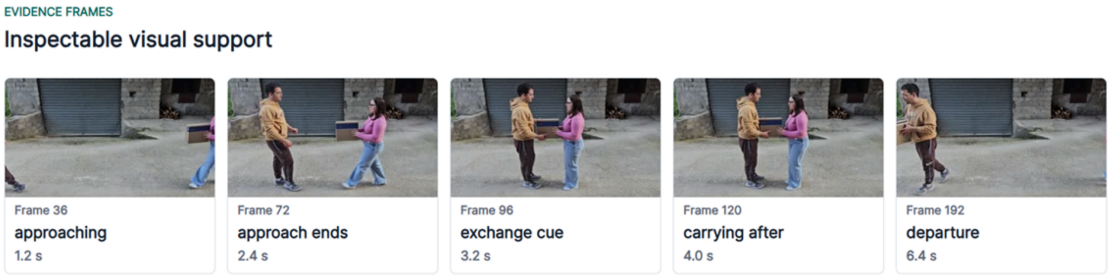


Figura 2.3: Un evento rilevato e i fotogrammi di evidenza che lo sostengono: ogni rilevazione si può ripercorrere fino alle osservazioni che l'hanno prodotta. Schermata di WATCHOUT; la stessa sequenza compare nel paper VIS MODE [8].

C'è una ragione precisa per non affidare al modello anche il ragionamento temporale. Bellodi et al. [14] misurano la capacità dei grandi modelli linguistici di ragionare in logica temporale a intervalli, e trovano che anche i modelli più recenti sbagliano in modo sistematico su relazioni che per un motore simbolico sono banali. Chiedere a un VLM "è avvenuto un passaggio di oggetto?" significa chiedergli di fare in un colpo solo percezione e inferenza temporale, e la seconda è proprio la parte su cui non è affidabile. Tenerla fuori dal modello non è una limitazione ma una scelta di progetto, e ha il vantaggio già discusso: la rilevazione resta una query, quindi resta verificabile.

Lo stesso schema si applica all'audio. Le reti pre-addestrate come **PANNs** [6] classificano segmenti sonori in centinaia di classi, e i modelli audio-linguaggio come **Qwen2-Audio** [15] rispondono in linguaggio naturale a domande su una registrazione, riconoscendo suoni ambientali come spari, frenate, vetri rotti. In entrambi i casi l'uscita si riconduce a un intervallo con un'etichetta, e a quel punto un intervallo sonoro e un intervallo visivo sono la stessa cosa per il livello degli eventi. Un sistema *multimodale* in questo senso non ha bisogno di fondere le modalità nel modello: le fonde nella tabella, e un evento può combinare uno sparo udito e una fuga vista con lo stesso operatore temporale che combina due intervalli visivi.

Il quadro che emerge è quindi questo: la letteratura recente ha reso il livello percettivo molto più potente e molto più facile da estendere, ma ha lasciato invariata la struttura a livelli, e ha semmai rafforzato l'argomento per tenere gli eventi al di

fuori del modello. Ciò che non è cambiato è che *qualcuno deve ancora scrivere le query.*

2.5 Il punto di partenza: WATCHOUT

Il sistema da cui parte questa tesi è **WATCHOUT**, un framework di sorveglianza forense multimodale sviluppato da Ghiffary Rifqialdi per la sua tesi di laurea magistrale [16], nel gruppo di ricerca del relatore e nell'ambito di una collaborazione con il gruppo di Research & Development di Hitachi, Tokyo. È un'istanza completa dello schema descritto in questo capitolo, e viene presentato qui come tale: questa sezione descrive il sistema *come si presentava all'inizio del lavoro*, mentre il Capitolo 5 descrive ciò che è diventato.

Il livello percettivo è duplice. Sul lato visivo, un modello visione-linguaggio descrive i fotogrammi campionati di un video con un vocabolario chiuso di relazioni tra persone, oggetti e veicoli, e un modulo di *re-identificazione* basato su una memoria di embedding visivi mantiene stabile l'identità degli attori tra un fotogramma e l'altro senza alcun addestramento. Sul lato sonoro, l'audio estratto dal video è classificato da PANNs o descritto da Qwen2-Audio, e le classi rilevanti per la sorveglianza, spari, esplosioni, stridori di pneumatici, vetri infranti, diventano intervalli sonori. Entrambi i flussi finiscono in un database **SQLite**: relazioni visive per fotogramma, consolidate in intervalli, e intervalli audio, ciascuno con i propri partecipanti e i frame di inizio e fine.

Sopra questa base sta il livello degli eventi, ed è qui che WATCHOUT è fedele alla lezione di ISEQL: *ogni evento è una query SQL.* Sedici in tutto: sei visivi (la rissa, lo sparo o l'esplosione, il passaggio di un oggetto, la sosta sospetta presso un veicolo, la collisione, la fuga in veicolo), quattro sonori (gli stessi meno il passaggio di oggetto e la sosta sospetta, che non hanno correlato acustico) e sei multimodali, che uniscono le rilevazioni delle due modalità. Gli operatori temporali sono quelli di ISEQL, resi come funzioni che generano il predicato SQL corrispondente. Non c'è alcun classificatore che decida se un evento è avvenuto: il risultato è l'insieme delle righe che soddisfano la query, e ognuna si può ricondurre agli intervalli e ai fotogrammi che l'hanno prodotta.

Il sistema è stato valutato in un **esperimento di ablazione a tre condizioni**: solo video (A), solo audio (B), e la loro unione (C), in cui le rilevazioni delle due modalità si sommano senza join temporali tra di esse. Il banco di prova è un dataset di trenta scene generate con Seedance [17], un generatore di video, e annotate con verità di riferimento per fotogramma; la Sezione 6.3 lo descrive. Con la combi-

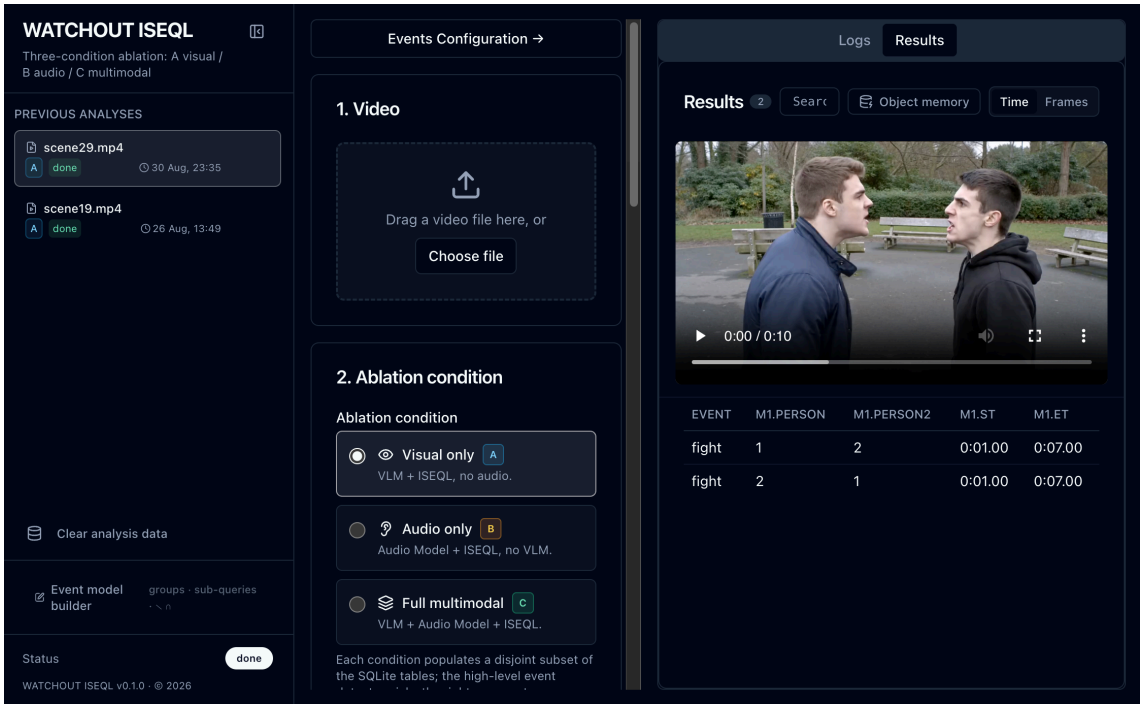


Figura 2.4: L'interfaccia di analisi di WATCHOUT: a sinistra le analisi precedenti, al centro il caricamento del video e la scelta della condizione, a destra i risultati, qui un evento *fight* tra due persone.

nazione migliore, un VLM commerciale per il video e Qwen2-Audio per l'audio, il sistema rileva tutti e trenta gli eventi attesi con quattro falsi positivi, e la condizione multimodale non è mai peggiore della migliore delle due condizioni unimodali.

WATCHOUT funziona, quindi, ed è verificabile per costruzione. Il suo limite, all'inizio di questo lavoro, era esattamente quello con cui si è chiuso il paragrafo precedente. Le sedici query erano *scritte a mano* nel codice Python del backend, con le soglie temporali fissate come valori di default degli argomenti; gli operatori temporali esistevano come funzioni che generano il predicato SQL corrispondente, ma chiamarle significava comunque scrivere una query. Aggiungere un diciassettesimo evento voleva dire modificare il sorgente e riavviare il servizio: chi usava il sistema poteva scegliere tra gli eventi previsti, non definirne uno proprio. È il problema che il resto della tesi affronta.

Capitolo 3

Query a intervalli per la modellazione di eventi

Il Capitolo 2 ha mostrato che un evento complesso è una relazione temporale tra intervalli, e ha anticipato un operatore e un'espressione. Questo capitolo dà a quell'idea la sua forma completa. Parte dalle relazioni tra intervalli di Allen, che sono il vocabolario di base; definisce ISEQL, il modello dei dati e gli otto operatori con le loro soglie, nella forma esatta che il sistema del Capitolo 5 compila in SQL; mostra come un'espressione dell'algebra si esegue su una base di dati relazionale; e infine presenta lo strumento che permette di scrivere quelle espressioni disegnando, e gli eventi di WATCHOUT riscritti in questa notazione, che sono la specifica che il resto della tesi deve soddisfare.

3.1 Intervalli e relazioni temporali

3.1.1 L'algebra di Allen

Un **intervallo** è una coppia di istanti $[T_s, T_e]$ con $T_s \leq T_e$. Nel nostro caso gli istanti sono indici di fotogramma, quindi numeri interi, e gli estremi sono inclusi: l'intervallo $[30, 90]$ contiene i fotogrammi dal trentesimo al novantesimo compresi. Questo dettaglio, che nel continuo non conta, nel discreto conta molto, e ci torneremo.

Allen [2] ha osservato che, dati due intervalli r e s, le posizioni reciproche possibili dei quattro estremi sono esattamente **tredici**, e che sono mutuamente esclusive ed esaustive: due intervalli qualsiasi stanno in una e una sola di queste relazioni. Sei sono relazioni "in avanti", sei sono le loro inverse, e la tredicesima, l'uguaglianza, è inversa di se stessa. La Tabella 3.1 le elenca con la condizione formale sugli estremi.

Relazione	Grafico	Notazione	Condizione	Inversa
before		$r \prec s$	$r.T_e < s.T_s$	after
meets		$r\,\mathrm{m}\,s$	$r.T_e = s.T_s$	met-by
overlaps		$r\,\mathrm{o}\,s$	$r.T_s < s.T_s < r.T_e < s.T_e$	overlapped-by
starts		$r\,\mathrm{s}\,s$	$r.T_s = s.T_s \wedge r.T_e < s.T_e$	started-by
during		$r\,\mathrm{d}\,s$	$s.T_s < r.T_s \wedge r.T_e < s.T_e$	contains
finishes		$r\,\mathrm{f}\,s$	$s.T_s < r.T_s \wedge r.T_e = s.T_e$	finished-by
equals		$r = s$	$r.T_s = s.T_s \wedge r.T_e = s.T_e$	(se stessa)

Tabella 3.1: Le tredici relazioni di Allen tra due intervalli r e s: le sei relazioni dirette, le loro inverse e l'uguaglianza. Nel grafico r è la barra scura e s quella chiara; le linee punteggiate indicano estremi che coincidono. L'inversa si ottiene scambiando le due barre.

L'algebra di Allen nasce per il ragionamento qualitativo: dato che r precede s e s è contenuto in t, cosa si può dire di r e t? Per la rilevazione di eventi ci interessa una domanda più semplice, ma con un'aggiunta che Allen non ha: date due tuple di M, sono nella relazione richiesta, *e a che distanza*?

3.1.2 Perché le tredici relazioni non bastano

Le relazioni di Allen hanno due caratteristiche che le rendono inadatte, così come sono, a definire eventi su dati percettivi.

La prima è che sono **esatte**. *Meets* richiede che r finisca nel preciso istante in cui s comincia. Un passaggio di oggetto reale è proprio questo, una persona smette di portare il pacco nell'istante in cui l'altra comincia; ma il livello percettivo non restituisce mai due intervalli che si toccano al fotogramma: restituisce un intervallo che finisce a 90, un altro che comincia a 96, con sei fotogrammi in cui il modello non ha saputo dire chi tenesse il pacco. Con le relazioni di Allen quella coppia è *before*, non *meets*, e l'evento non viene rilevato. Serve una **tolleranza**.

La seconda è che sono **senza distanza**. *Before* vale per due intervalli a due secondi di distanza come per due intervalli a due ore di distanza. Ma un evento come "uno sparo seguito da una fuga" ha senso solo se la fuga segue lo sparo entro pochi secondi: la vicinanza temporale è parte della definizione. Serve una **soglia**.

C'è poi una questione che nasce dal discreto. Su istanti continui, due intervalli che si toccano ($r.T_e = s.T_s$) non condividono durata. Su fotogrammi a estremi

inclusi, $[0, 50]$ e $[50, 100]$ condividono il fotogramma 50: durante quel fotogramma entrambi i predicati valgono, quindi i due intervalli si sovrappongono, per quanto poco. Se "prima" debba ammettere o no il fotogramma condiviso è una scelta, non un fatto, e va resa esplicita nella definizione dell'operatore. ISEQL lo fa con un parametro; il Capitolo 5 racconta quale valore il sistema adotta e perché.

3.2 ISEQL

ISEQL, *Interval-based Surveillance Event Query Language* [1, 9], risponde a queste esigenze estendendo l'algebra relazionale con operatori di join in cui la condizione è una relazione temporale tra intervalli, resa tollerante e limitata da soglie.

3.2.1 Il modello dei dati

Il linguaggio opera su relazioni con uno schema fisso. Una relazione di intervalli ha attributi

$$(pred,\ arg_1, \ldots, arg_n,\ T_s,\ T_e), \tag{3.1}$$

dove *pred* è il nome del predicato di medio livello, arg_i sono i partecipanti in posizione, e T_s, T_e gli estremi. Il numero di argomenti dipende dal predicato: *running* ne ha uno, *carrying* due (chi porta, cosa), *load_object_vehicle* tre. È la relazione M dell'equazione (2.2), con gli attributi chiamati come nelle query: *pred* per il predicato P_i e $arg_1, arg_2, \ldots$ per gli argomenti $a_1, a_2, \ldots$.

Come osservato nella Sezione 2.2, una relazione con questo schema è **denormalizzata**: le colonne arg_i sono tante quante ne servono al predicato con più argomenti, e nelle altre tuple quelle in eccesso restano vuote. WATCHOUT conserva invece gli intervalli in forma normalizzata, e la Sezione 3.3 mostra che proprio questa differenza è il primo problema da risolvere per l'integrazione.

Un'espressione ISEQL nomina le occorrenze della relazione con **alias**, tradizionalmente $M_1, M_2, \ldots$, come si fa in SQL con le copie di una tabella in un self-join. Ogni alias è selezionato per predicato e, se serve, per classe degli argomenti:

$$\sigma_{pred=\text{“carrying”} \wedge arg_1=\text{“person”} \wedge arg_2=\text{“object”}}(M_1). \tag{3.2}$$

La selezione sul predicato è la porta d'ingresso di ogni query: sceglie *quali* intervalli entrano nel ragionamento, prima che gli operatori temporali dicano *come* devono stare tra loro.

3.2.2 Gli otto operatori temporali

ISEQL definisce otto operatori di join temporale. Ciascuno prende due relazioni, r a sinistra e s a destra, e restituisce le coppie di tuple che soddisfano la relazione, entro le soglie. Le soglie sono due: δ vincola la distanza tra gli inizi (o tra la fine di r e l'inizio di s, per *Before*), ε vincola la distanza tra le fini. La Tabella 3.2 li definisce tutti; le definizioni sono quelle del sistema, prese dal modulo che le rende in SQL, e coincidono con quelle pubblicate [9].

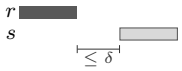
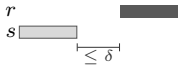
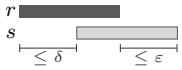
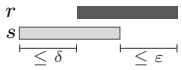
Operatore	Soglie	Grafico	Condizione su r e s
Bef (Before)	δ		$r.T_e \leq s.T_s \ \wedge \ s.T_s - r.T_e \leq \delta$
Aft (After)	δ		s Bef$_\delta$ r
SP (Start Preceding)	δ		$r.T_s \leq s.T_s \leq r.T_e \ \wedge \ s.T_s - r.T_s \leq \delta$
EF (End Following)	ε		$r.T_s \leq s.T_e \leq r.T_e \ \wedge \ r.T_e - s.T_e \leq \varepsilon$
DJ (During Join)	δ, ε		$s.T_s \leq r.T_s \ \wedge \ r.T_e \leq s.T_e$ $r.T_s - s.T_s \leq \delta \ \wedge \ s.T_e - r.T_e \leq \varepsilon$
RDJ (Reverse During)	δ, ε		s DJ$_{\delta,\varepsilon}$ r
LOJ (Left Overlap)	δ, ε		$r.T_s \leq s.T_s \leq r.T_e \leq s.T_e$ $s.T_s - r.T_s \leq \delta \ \wedge \ s.T_e - r.T_e \leq \varepsilon$
ROJ (Right Overlap)	δ, ε		s LOJ$_{\delta,\varepsilon}$ r

Tabella 3.2: Gli otto operatori temporali di ISEQL. Le disuguaglianze sono nella forma non stretta ($\leq$); la Sezione 3.2.4 mostra come ISEQL+ le parametrizza. Una soglia omessa vale ∞. Nel grafico r è la barra scura e s quella chiara; le quote indicano le distanze limitate da δ ed ε, e il tratteggio un estremo su cui l'operatore non pone alcun vincolo.

Conviene leggere la tabella a gruppi, perché gli otto operatori non sono otto idee diverse.

Before e After sono la relazione di sequenza: r finisce, poi s comincia, con un intervallo vuoto di al più δ tra i due. *After* è *Before* con gli argomenti scambiati. Insieme coprono le relazioni *before*, *meets* e le loro inverse di Allen: *meets* è il caso $\delta = 0$.

During e Reverse During sono il contenimento: in DJ, r è contenuto in s, e δ, ε limitano di quanto s sporge da r all'inizio e alla fine. Con $\delta = \varepsilon = 0$ si ottiene

equals; con una sola delle due a zero, *starts* o *finishes*. Il contenimento è la relazione degli eventi "mentre": una persona che corre *mentre* un allarme suona.

Left Overlap e Right Overlap sono la sovrapposizione parziale: r comincia prima, s finisce dopo, e i due condividono un tratto. È la relazione degli eventi che si passano il testimone, come una persona che si avvicina a un veicolo e poi vi entra mentre è ancora vicina.

Start Preceding ed End Following sono diversi dagli altri sei: non fissano tutti e quattro gli estremi, ma solo due. SP dice che s comincia durante r, entro δ dal suo inizio, e non dice nulla su quando finisce; EF dice che s finisce durante r, entro ε dalla sua fine, e non dice nulla su quando comincia. Sono quindi *meno specifici*: SP è soddisfatto sia da una coppia LOJ sia da una coppia RDJ. Servono quando l'evento richiede un vincolo su un solo estremo, come "un veicolo parte poco dopo che una persona vi è salita", in cui conta quando la partenza *comincia* rispetto alla salita, non quando finisce.

Questa struttura ha una conseguenza pratica per chi costruisce uno strumento di modellazione: dati due intervalli disegnati, sei degli otto operatori si possono *riconoscere* dalla geometria, perché le loro condizioni coprono tutte le disposizioni possibili e, dove due di esse valgono insieme, una è più specifica dell'altra. SP ed EF invece non si riconoscono mai da soli, perché sono implicati da qualcuno degli altri sei e non sono mai il caso più specifico: vanno scelti esplicitamente. Il Capitolo 5 usa proprio questa proprietà.

3.2.3 Selezione, proiezione e operazioni insiemistiche

Gli operatori temporali producono coppie (o triple, o n-uple) di intervalli. Il resto dell'algebra relazionale serve a vincolarle e a decidere cosa restituire.

La **selezione** σ compare in due ruoli. Sul singolo alias sceglie predicato e classi degli argomenti, come nell'equazione (3.2), oppure vincola un attributo dell'intervallo contro una costante, come $\sigma_{M_1.duration \geq 3}$ per una presenza che duri almeno tre secondi. Sul risultato di un join esprime **condizioni incrociate** tra alias, che sono ciò che lega i partecipanti: $\sigma_{M_1.arg_1 = M_2.arg_1}$ richiede che sia la *stessa* persona a correre e a salire sul veicolo; $\sigma_{M_1.arg_1 \neq M_2.arg_1 \wedge M_1.arg_2 = M_2.arg_2}$ richiede due persone diverse e lo stesso oggetto. Senza le condizioni incrociate l'operatore temporale accoppierebbe qualunque *carrying* con qualunque altro, e il passaggio di oggetto diventerebbe "due persone qualsiasi portano due oggetti qualsiasi in sequenza".

La **proiezione** π sceglie le colonne del risultato, e per un evento la scelta ha un significato preciso: chi sono i partecipanti dell'evento composto, e qual è il suo intervallo. Per il passaggio di oggetto, $\pi_{M_1.arg_1, M_2.arg_1, M_1.arg_2, M_1.T_s, M_2.T_e}$ restituisce

chi dà, chi riceve, cosa, e un intervallo che va dall'inizio del primo trasporto alla fine del secondo. L'evento composto è a sua volta un intervallo con partecipanti, cioè una tupla dello stesso schema (3.1): è questo che permette di comporre eventi di eventi.

Le **operazioni insiemistiche** $\cup$, $\setminus$, $\cap$ combinano espressioni complete. L'unione è il costrutto multimodale per eccellenza: l'evento *fight* è l'unione tra una rissa vista (un intervallo *physical_altercation* tra due persone) e una rissa udita (un grido seguito entro cinque secondi da un impatto). La differenza esprime le eccezioni, "un oggetto lasciato a terra *e nessuno* che lo riprende". Infine la **disgiunzione** $\vee$ tra operatori ammette che una coppia soddisfi uno qualsiasi di più operatori, $\text{Bef}_\delta \vee \text{LOJ}$: serve quando la sequenza e la sovrapposizione sono entrambe accettabili, come in una fuga in veicolo in cui la corsa può finire un attimo prima o un attimo dopo che la persona ha aperto la portiera.

3.2.4 ISEQL+: strettezza e robustezza

L'estensione ISEQL+ [11] aggiunge a ciascun operatore tre parametri che rendono esplicite scelte che in ISEQL erano fissate nella definizione.

Gli operatori di **strettezza** $\zeta, \eta \in \{\leq, <\}$ sostituiscono le disuguaglianze non strette della Tabella 3.2: ζ agisce sul confronto tra gli inizi (o tra fine e inizio, per *Before*), η su quello tra le fini. È esattamente il modo di rendere esplicita la scelta sul fotogramma condiviso discussa nella Sezione 3.1: con $\zeta = <$, *Before* rifiuta due intervalli che si toccano; con $\zeta = \leq$ li accetta.

Il parametro di **robustezza** $\rho \geq 0$ allarga ogni confronto di ρ fotogrammi, in modo da tollerare piccole imprecisioni sugli estremi senza cambiare l'operatore. Con tutti i parametri, *Before* diventa

$$r \ \text{Bef}_{\delta;\,\zeta,\rho} \ s \quad \Longleftrightarrow \quad r.T_e \ \zeta \ (s.T_s + \rho) \ \wedge \ s.T_s - r.T_e \leq \delta + \rho, \tag{3.3}$$

e le altre sette definizioni si estendono allo stesso modo: ogni confronto tra estremi riceve ρ dalla parte che lo rende più permissivo, e ogni soglia si allarga di ρ. La notazione che il sistema usa scrive tutti i parametri in chiaro, ad esempio $\text{Bef}(\delta{:}5;\ \zeta{:}\leq,\ \rho{:}0)$, con ∞ per una soglia assente.

Nel resto della tesi si usa questa forma completa. Vale la pena notare che i parametri non sono ornamenti: nel Capitolo 5 si vedrà che la strettezza di *Before* è stata l'oggetto di una decisione presa, cambiata, e infine allineata tra l'editor e le query, e che averla come parametro esplicito è ciò che ha reso la discussione possibile.

3.2.5 Dall'algebra al DBMS

Un'espressione ISEQL è un'espressione di algebra relazionale, e come tale dice quali tuple devono comparire nel risultato, non come calcolarle. Per eseguirla con un sistema di gestione di basi di dati (DBMS) le strade sono due, e si distinguono per un aspetto preciso: se il motore sa, o non sa, di stare confrontando intervalli.

La prima strada è **tradurre** l'espressione nel linguaggio che il DBMS già capisce, cioè SQL. Selezione, proiezione, unione e differenza hanno un corrispondente diretto, e un operatore temporale diventa un join la cui condizione è la disuguaglianza della Tabella 3.2, scritta sulle colonne degli estremi. Il motore non sa nulla di intervalli: vede un join con una condizione di disuguaglianza e lo valuta con gli algoritmi generali di cui dispone. Il vantaggio è che non serve alcun componente nuovo, e che la query resta un testo che si può leggere ed eseguire a mano. Il limite è il costo. Un join su disuguaglianze non può sfruttare un indice come un join su uguaglianza, e nel caso peggiore il motore confronta ogni intervallo di M_1 con ogni intervallo di M_2, con costo quadratico nel numero di intervalli.

La seconda strada è **implementare gli operatori** come algoritmi di join dedicati, che sanno di lavorare su intervalli. È quella seguita da Piatov et al. [3], che in un lavoro su rivista mostrano come tutti gli operatori della Tabella 3.2 si possano valutare con una **scansione del piano** (*plane sweep*). Gli estremi degli intervalli delle due relazioni vengono ordinati una volta sola e percorsi in ordine di tempo, come una linea che scorre lungo l'asse temporale. Quando la linea incontra l'inizio di un intervallo, questo entra nell'insieme degli intervalli *attivi*; quando ne incontra la fine, ne esce. Le coppie del risultato si cercano confrontando ogni intervallo con quelli attivi dell'altra relazione, invece che con tutti, e la memoria contiene in ogni istante solo gli intervalli attivi. Gli autori estendono questo schema ai predicati con soglie di ISEQL e curano l'organizzazione in memoria degli intervalli attivi in modo da sfruttare la cache del processore: il costo diventa vicino a lineare, e i loro algoritmi risultano più veloci di diverse volte rispetto ai join relazionali. Il repository di WATCHOUT contiene già un esecutore di questo tipo.

Questa tesi segue, per ora, la prima strada, per tre ragioni. La prima è che WATCHOUT esegue già i suoi sedici eventi come query SQL, e tradurre verso lo stesso bersaglio permette di confrontare le occorrenze di un evento compilato con quelle della query scritta a mano, che è il criterio di correttezza della Sezione 6.2.1. La seconda è la verificabilità: la query generata resta visibile e leggibile, come richiede l'uso forense. La terza è la dimensione dei dati: un'analisi riguarda un video alla volta, con al più qualche centinaio di intervalli, e su questi numeri il costo quadratico non si vede. Il passaggio all'implementazione efficiente è il passo

successivo, e il sistema è costruito per accoglierlo: il registro degli eventi conserva il modello dell'evento, non il SQL, che viene generato solo al momento della rilevazione. Cambiare esecutore significa quindi cambiare il bersaglio della compilazione, non gli eventi; la Sezione 7.3 torna su questo punto, mentre la sezione seguente descrive la traduzione in SQL adottata.

3.2.6 Dall'algebra al SQL

Nella strada adottata, un'espressione ISEQL non richiede un motore dedicato: si traduce in SQL standard su un database relazionale, ed è così che WATCHOUT la esegue. La traduzione segue la struttura dell'espressione.

Ogni alias M_i con la sua selezione sul predicato diventa una *common table expression* che estrae dalla tabella degli intervalli le righe di quel predicato con le colonne rinominate. Un operatore temporale tra due alias diventa un join tra le due CTE, la cui condizione è la disuguaglianza della Tabella 3.2 scritta sulle colonne degli estremi. *Before* con $\delta = 2$ secondi, per esempio, si rende come

```
M1.et <= M2.st AND (M2.st - M1.et) <= 2
```

e ogni altro operatore è una congiunzione analoga. Le condizioni incrociate vanno nella clausola `WHERE`, la proiezione nella `SELECT`, l'unione in `UNION`. È una traduzione meccanica, e infatti nel Capitolo 5 è affidata a un compilatore.

Ogni join di questo tipo ha il costo quadratico discusso nella Sezione 3.2.5. Per un'analisi forense su un singolo video, con centinaia di intervalli, il costo è trascurabile e la semplicità del SQL vale più dell'efficienza; quando gli intervalli diventano milioni, come in un archivio di mesi, è il momento della scansione del piano.

3.3 Modellare eventi senza scrivere query

Tutto ciò che precede presuppone che qualcuno scriva l'espressione. Il framework interattivo di Persia et al. [10] aveva già posto il problema, esponendo i tre livelli del sistema attraverso un'interfaccia che permette di creare definizioni di eventi durante l'uso. Ma un'interfaccia che chiede di scrivere σ e π sposta il problema, non lo risolve: chi controlla le telecamere non conosce l'algebra relazionale, e non deve conoscerla.

3.3.1 L'ISEQL Event Model Builder

L'**ISEQL Event Model Builder**, sviluppato in una tesi precedente nello stesso gruppo, prende la strada opposta: l'utente non scrive l'espressione, la *disegna*. È un'applicazione desktop in Python di circa millecinquecento righe, con una sola dipendenza esterna, e con una separazione netta tra tre componenti che vale la pena distinguere perché ricompaiono, trasformate, nel sistema del Capitolo 5.

Il **modello** è una lista di intervalli. Ogni intervallo ha un predicato con i suoi argomenti tipizzati, *carrying(person, object)*, e due coordinate T_s e T_e su una linea temporale astratta. Le coordinate non sono fotogrammi di un video: sono le posizioni in cui l'utente ha disegnato le barre, e servono solo a esprimere una relazione. Il modello può contenere condizioni incrociate tra intervalli e si salva in un file JSON.

Il **rilevatore di operatori** è la parte che rende il disegno significativo. Date due barre, decide quale operatore della Tabella 3.2 descrive la loro posizione reciproca, e ricava le soglie dalla geometria: se r finisce a 40 e s comincia a 50, il rilevatore risponde Bef con $\delta = 10$. La decisione segue l'ordine di specificità discusso nella Sezione 3.2.2:

$$\mathrm{op}(r,s) = \begin{cases} \text{Bef}, & r.T_e \le s.T_s \\ \text{Aft}, & s.T_e \le r.T_s \\ \text{DJ}, & s.T_s \le r.T_s \wedge r.T_e \le s.T_e \\ \text{RDJ}, & r.T_s \le s.T_s \wedge s.T_e \le r.T_e \\ \text{LOJ}, & r.T_s \le s.T_s < r.T_e \le s.T_e \\ \text{ROJ}, & s.T_s \le r.T_s < s.T_e \le r.T_e \end{cases} \tag{3.4}$$

con le soglie δ ed ε pari alle distanze tra gli estremi corrispondenti. I casi sono valutati nell'ordine e il primo che vale vince: è così che il contenimento con estremi coincidenti viene letto come DJ e non come LOJ. SP ed EF non compaiono, per la ragione già detta: non sono mai il caso più specifico, quindi l'utente li sceglie a mano quando vuole vincolare un solo estremo, e il rilevatore ricava allora l'unica soglia che quell'operatore prende.

Il **generatore dell'espressione** percorre gli intervalli in ordine e scrive il testo ISEQL: una selezione per ciascun alias, l'operatore rilevato tra ogni coppia adiacente con le soglie in chiaro, le condizioni incrociate in una selezione esterna e la proiezione in testa. Il risultato è un'espressione nella notazione Unicode dell'algebra, σ, π, δ, ε, pronta per essere letta da una persona.

3.3.2 Cosa il Builder non faceva

Il Builder si ferma lì. Non esegue nulla: non contiene una connessione a un database, un cursore, una riga di SQL. Produce la definizione di un evento e la lascia a chi la userà. Questo era il suo scopo, e non è una mancanza; ma per farne il livello di modellazione di WATCHOUT restavano da colmare alcune distanze, che conviene elencare perché sono il programma di lavoro del Capitolo 5.

La prima riguarda il **modello dei dati**. Il Builder assume la relazione piatta dell'equazione (3.1), con argomenti posizionali arg_1, arg_2. WATCHOUT invece normalizza: un intervallo sta in una tabella e i suoi partecipanti in un'altra, come insieme non ordinato discriminato solo dalla classe. Non esiste una colonna "posizione". Per far incontrare i due mondi serve una regola che ricavi la posizione dal tipo.

La seconda riguarda l'**espressività**. Confrontando i sedici eventi di WATCHOUT con ciò che il Builder sapeva rappresentare, mancavano quattro costrutti: un intervallo che accetti *più predicati* nello stesso posto, dato che uno sparo visibile e un'esplosione visibile sono due predicati distinti per lo stesso evento; un vincolo su *un attributo di un singolo intervallo*, come una durata minima o una confidenza minima; una condizione di *conteggio*, come "almeno due persone" per una rissa; e la *disgiunzione tra operatori*, Bef $\vee$ LOJ, che tre eventi richiedono. Nessuno di questi è una relazione temporale, e infatti il Builder, che di relazioni temporali si occupava, non li aveva.

La terza riguarda le **soglie**. Nel Builder la soglia è la distanza disegnata: $\delta = 10$ perché le barre distano dieci unità. Ma un evento reale ha soglie con un significato, "entro due secondi", "per almeno tre secondi", che chi modella vuole scegliere e nominare, non ricavare dalla posizione delle barre. Il disegno deve esprimere *quale* relazione, non *quanto* stretta.

La quarta, infine, è la più semplice da dire: l'espressione va **eseguita**. Serve un compilatore da ISEQL a SQL, un posto dove salvare i modelli, e un modo perché il servizio di analisi li trovi accanto agli eventi predefiniti.

3.4 Gli eventi di WATCHOUT in ISEQL

Prima di costruire il compilatore, i sedici eventi di WATCHOUT sono stati riscritti uno per uno nella notazione del Builder. Il documento che ne è risultato ha un ruolo preciso in questa tesi: è la **specifica** che l'integrazione deve soddisfare. Se il compilatore è corretto, dare in pasto queste espressioni deve produrre esattamente le occorrenze che le query scritte a mano già producono, e la Sezione 6.2.1 mostra

che è così. Ne riportiamo quattro, scelte per coprire i costrutti del linguaggio dal più semplice al più ricco. La notazione è quella completa di ISEQL+, con tutti i parametri in chiaro; `st` ed `et` sono gli estremi in secondi.

Un intervallo con un vincolo di attributo. La sosta sospetta presso un veicolo non ha operatori temporali: è un solo intervallo, con un vincolo sulla sua durata.

```
π_{M1.arg1, M1.arg2, M1.st, M1.et} (
  σ_{(M1.et - M1.st) ≥ 3} (
    σ_{pred="suspicious_near_vehicle"
        ∧ arg1="person" ∧ arg2="vehicle"}(M1)))
```

La selezione interna sceglie gli intervalli, quella esterna li filtra per durata, la proiezione restituisce la persona, il veicolo e l'intervallo stesso.

Due intervalli, una soglia, la stessa persona. La fuga in veicolo è una corsa seguita, entro due secondi, dall'ingresso in un veicolo, e la persona deve essere la stessa.

```
π_{M1.arg1, M2.arg2, M1.st, M2.et} (
  σ_{M1.arg1 = M2.arg1} (
    σ_{pred="running" ∧ arg1="person"}(M1)
    Bef(δ:2; ζ:≤, ρ:0)
    σ_{pred="enter_or_exit_vehicle"
        ∧ arg1="person" ∧ arg2="vehicle"}(M2)))
```

È l'equazione (2.3) in forma concreta: l'operatore con la soglia in secondi, la condizione incrociata che lega i due alias, e una proiezione che costruisce l'intervallo dell'evento composto dall'inizio della corsa alla fine dell'ingresso.

Due intervalli, soglia infinita, persone diverse. Il passaggio di oggetto è la formula (2.4) del Capitolo 2, con una scelta in più: la soglia è ∞, perché nei dati di WATCHOUT il passaggio può essere diretto o indiretto, con l'oggetto posato e ripreso più tardi.

```
π_{M1.arg1, M2.arg1, M1.arg2, M1.st, M2.et} (
  σ_{M1.arg1 ≠ M2.arg1 ∧ M1.arg2 = M2.arg2} (
    σ_{pred="carrying" ∧ arg1="person" ∧ arg2="object"}(M1)
    Bef(δ:∞; ζ:≤, ρ:0)
    σ_{pred="carrying" ∧ arg1="person" ∧ arg2="object"}(M2)))
```

Le due condizioni incrociate sono ciò che distingue un passaggio di oggetto da due trasporti qualsiasi: persone diverse, stesso oggetto.

Un evento multimodale. La rissa unisce due espressioni indipendenti: una sul canale visivo, un intervallo *physical_altercation* tra due persone distinte, e una sul canale sonoro, un grido seguito entro cinque secondi da un impatto.

```
π_{M1.st, M2.et} (
  σ_{pred="shout"}(M1)
  Bef(δ:5; ζ:≤, ρ:0)
  σ_{pred="impact"}(M2))
∪
π_{M1.arg1, M1.arg2, M1.st, M1.et} (
  σ_{M1.arg1 ≠ M1.arg2} (
    σ_{pred="physical_altercation"
        ∧ arg1="person" ∧ arg2="person"}(M1)))
```

L'unione è tutto il meccanismo multimodale: le due metà non sanno l'una dell'altra, e il risultato contiene le risse viste e quelle udite. È la scelta di progetto di WATCHOUT discussa nella Sezione 2.5, e in ISEQL costa un solo simbolo.

Questi quattro esempi contengono, tra tutti, ogni costrutto elencato nella Sezione 3.3: la selezione su predicato e classi, il vincolo su un attributo, l'operatore con soglia finita e infinita, le condizioni incrociate di uguaglianza e disuguaglianza, la proiezione che definisce l'evento composto, l'unione. Il Capitolo 6 verifica il sistema su tutti e sedici, confrontando le occorrenze che produce con quelle delle query scritte a mano.

Capitolo 4

Tecnologie utilizzate

Il sistema è composto da due servizi, un backend Python che espone un'API HTTP e un frontend web che la consuma, e da un database SQLite in cui il backend legge e scrive. La Figura 4.1 mostra come le parti sono collegate e la Tabella 4.1 riassume i componenti; le sezioni che seguono descrivono ciascuno con il ruolo che ha nell'integrazione. Le scelte di linguaggio, database e framework sono di WATCHOUT; quelle relative all'editor e al compilatore sono di questa tesi e vengono motivate dove compaiono.

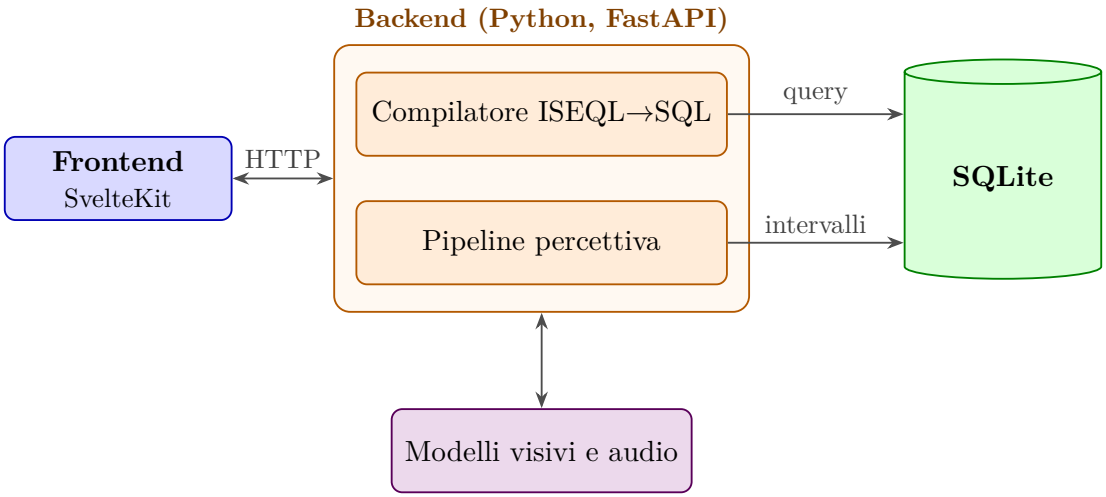


Figura 4.1: Le parti del sistema e i loro collegamenti. Il frontend parla con il backend via HTTP; il backend scrive gli intervalli prodotti dai modelli nel database SQLite e vi esegue le query generate dal compilatore.

Livello	Tecnologia	Ruolo nel sistema
Database	SQLite (modalità WAL)	intervalli percettivi, registro degli eventi
Backend	Python $\geq$ 3.11	pipeline percettiva, nucleo del Builder, compilatore ISEQL→SQL
	FastAPI, Pydantic, uvicorn	API HTTP, API di analisi tipizzata, SSE
	pytest	test di equivalenza e di compilazione
Frontend	Svelte 5, SvelteKit, TypeScript	interfaccia di analisi, editor a linea temporale
	Vite, Vitest	build e test del rilevatore di operatori
	bits-ui, Tailwind CSS	componenti e stile
Percezione	Gemini, Ministral (API)	descrizione dei fotogrammi
	SigLIP + ChromaDB	re-identificazione dei partecipanti
	ffmpeg, PANNs, Qwen2-Audio	estrazione e classificazione dell'audio
Esecuzione	Docker Compose	un container per backend e uno per frontend

Tabella 4.1: I componenti del sistema e il loro ruolo.

4.1 Base di dati: SQLite

Tutti i dati del sistema stanno in un unico file SQLite, aperto dal backend con tre direttive:

```
PRAGMA journal_mode = WAL;
PRAGMA busy_timeout = 60000;
PRAGMA foreign_keys = ON;
```

La modalità *write-ahead log* consente letture concorrenti alla scrittura: un'analisi percettiva in corso inserisce intervalli mentre l'interfaccia interroga quelle già concluse. Il `busy_timeout` evita che una lettura fallisca durante un commit lungo; i vincoli di chiave esterna, disattivati per default in SQLite, sono attivati perché la tabella dei partecipanti riferisce quella degli intervalli.

Le tabelle rilevanti per l'integrazione sono quattro. `VisualPerInterval` contiene un intervallo visivo per riga (`RelationID`, `AnalysisID`, `RelationType`, `StartFrame`, `EndFrame`); `VisualParticipant` i suoi partecipanti, uno per riga, con `ClassID` e `Class` $\in$ {`person`, `car`, `vehicle`, `object`, ...}; `AudioPerInterval` gli intervalli sonori, con `AudioClass` e `Confidence` e senza partecipanti; `EventSpec` il registro degli eventi, predefiniti e definiti dall'utente. Le prime tre sono le tabelle su cui il SQL generato opera (Sezione 5.2.1).

Tre proprietà di SQLite sono state determinanti. Le **viste** sono relazioni virtuali

senza costo di memorizzazione, e permettono di presentare lo schema normalizzato nella forma piatta dell'equazione (3.1) senza duplicare i dati: è la forma che la prima versione del lavoro ha usato, prima che la corrispondenza passasse nei join generati dal compilatore (Sezione 5.2.1). Le **common table expression** (`WITH ... AS`) e `UNION` danno una traduzione uno-a-uno della struttura di un'espressione ISEQL: una CTE per alias, una condizione di join per operatore, una `UNION` per l'unione insiemistica. Infine un limite: `MIN(a, b)` e `MAX(a, b)` con due argomenti sono *funzioni scalari*, valutate riga per riga, e non aggregate; la Sezione 6.4 descrive il difetto che questa distinzione ha prodotto nell'aggregazione delle rilevazioni multimodali.

SQLite non ha indici su intervalli né join dedicati: un join con condizione di disuguaglianza come quello della Sezione 3.2.6 è valutato come prodotto tra le due CTE filtrato dalla condizione, con costo $O(|M_1| \cdot |M_2|)$. Per un'analisi, con al più qualche centinaio di intervalli, l'esecuzione resta sotto il decimo di secondo.

4.2 Backend: Python e FastAPI

Il backend è un'applicazione Python ($\geq$ 3.11) servita da `uvicorn`. Il nucleo del Builder (`interval.py`, `operator_detector.py`, `query_builder.py`, circa ottocento righe) vive nel pacchetto `service.impl.iseql`: è stato copiato per primo senza modifiche al di fuori dei percorsi di import, e le estensioni successive sono marcate nel sorgente, così che ciò che è ereditato resti distinguibile da ciò che è stato aggiunto; il compilatore ISEQL→SQL e le definizioni degli operatori stanno nel pacchetto `iseql` (`helpers.py`, `parser.py`, `compiler.py`). La scelta del linguaggio segue da questo: il Builder e la pipeline di WATCHOUT erano entrambi in Python, e il compilatore doveva chiamare il primo e produrre SQL per la seconda.

FastAPI espone i servizi come endpoint HTTP montati sotto `/api`. Quelli aggiunti dall'integrazione sono elencati nella Tabella 4.2. Le richieste e le risposte della pipeline di analisi sono dichiarate come modelli **Pydantic**, che FastAPI valida e da cui genera la specifica OpenAPI del servizio. Gli endpoint degli eventi ricevono invece il modello come documento JSON e lo validano nel compilatore, che è il posto dove la conoscenza necessaria sta: un modello mal formato non produce un errore di schema ma un `ModelValidationError` che nomina l'alias e il costrutto, restituito come errore 400 (Sezione 5.4).

Il Builder è arrivato nel browser in due passi. L'applicazione originale usava **pywebview**, cioè una finestra nativa che carica una pagina HTML e JavaScript e vi inietta un oggetto `window.pywebview.api` attraverso cui il codice della pagina

Metodo	Percorso	Funzione
POST	`/api/iseql/compile`	compila un'espressione ISEQL testuale nel modello e nel SQL
POST	`/api/iseql/model/compile`	compila il modello dell'editor in ISEQL e SQL, a ogni modifica
POST	`/api/iseql/preview`	rende un modello in testo ISEQL e riporta gli operatori tra gli intervalli
POST	`/api/iseql/complete`	completamento per l'editor testuale
GET	`/api/iseql/vocabulary`	predicati, classi e letture degli operatori
GET/POST	`/api/events`	elenco del registro degli eventi e creazione di un evento
GET/PUT/DELETE	`/api/events/{id}`	lettura, modifica ed eliminazione di un evento
GET	`/api/events/types`	eventi di una condizione con le loro soglie regolabili

Tabella 4.2: Gli endpoint aggiunti dall'integrazione e usati dall'editor.

chiama i metodi della classe Python `ISEQLAPI`. Tutte le chiamate passavano da una sola riga:

```
await window.pywebview.api[method](...args)
```

Nella prima versione dell'integrazione quella riga è stata sostituita con una `POST` a `/api/iseql/rpc`, che riceve nome del metodo e argomenti e li inoltra alla stessa classe: la logica di editing del Builder girava invariata dietro il server di WATCHOUT, con una sessione di editing per processo, come nell'applicazione desktop. L'editor attuale ha invertito il rapporto. Il modello dell'evento vive nel browser e il server non conserva alcuno stato di editing: a ogni modifica riceve il modello, lo compila e restituisce l'espressione ISEQL e il SQL (`/api/iseql/model/compile`), e il salvataggio scrive il modello nel registro (`/api/events`). L'endpoint `/api/iseql/rpc` e quelli di pubblicazione della prima versione restano nel repository, ma l'editor non li usa più.

Le operazioni lunghe, come l'analisi percettiva di un video, usano *server-sent events* (libreria `sse-starlette`): il backend risponde con un flusso di eventi e il frontend riceve le righe di log man mano. Un evento definito dall'utente viene eseguito dallo stesso endpoint degli eventi predefiniti, `/api/analysis/{id}/detect`.

I test sono in **pytest**, circa 750 nel backend. I più importanti per questa tesi sono quelli di equivalenza: per ciascun evento eseguono il modello compilato e un SQL di

riferimento sugli stessi database registrati, e confrontano gli insiemi di occorrenze, non il solo numero di righe.

4.3 Frontend: Svelte, SvelteKit e TypeScript

Il frontend è un'applicazione **SvelteKit** in **TypeScript**, compilata con Vite. Il componente `iseql-timeline.svelte` è l'editor a linea temporale; il componente `event-editor.svelte` lo contiene insieme alla vista testuale, e i pannelli per condizioni e proiezione si aprono dalla linea temporale. Il modello di evento lato client è definito in `iseql-model.ts`.

Svelte 5 gestisce lo stato con le *rune*. La runa `$state` dichiara un valore reattivo; `$derived` dichiara un valore calcolato, aggiornato quando le sue dipendenze cambiano. Il modello dell'editor è un insieme di gruppi di intervalli, e l'espressione ISEQL mostrata è derivata da esso:

```
let groups = $state<BuilderGroup[]>([]);
let expr = $derived(renderExpression(groups));
```

Trascinare una barra modifica `ts` e `te` dell'intervallo in `groups`; l'operatore tra due intervalli adiacenti e l'espressione si ricalcolano senza codice di propagazione. È il motivo per cui la linea temporale del Builder, che nell'originale aggiornava l'interfaccia con chiamate esplicite dopo ogni gesto, è stata riscritta e non trasposta.

I tipi del modello sono dichiarati in TypeScript e rispecchiano le classi Python del backend:

```
interface BuilderInterval {
  id: string; pred: string; args: string[]; ts: number; te: number;
}
interface BuilderOperator {
  op: string; // 'auto' | Bef | Aft | SP | EF | DJ | RDJ | LOJ | ROJ
  delta: string; epsilon: string; // numero, nome di parametro, o '' per
      infinito
  zeta: string; eta: string; rho: string;
}
interface BuilderGroup {
  name: string; intervals: BuilderInterval[]; ops: BuilderOperator[];
  projection: string[] | null; crossConditions: CrossCondition[];
}
```

Un campo `op` uguale a `'auto'` significa che l'operatore va rilevato dalla geometria; le soglie sono stringhe perché possono essere un numero, il nome di un parametro dichiarato o vuote per ∞. Il rilevatore di operatori dell'equazione (3.4) è

implementato in `iseql-model.ts` come funzione pura e testato con **Vitest** (45 casi in `iseql-model.test.ts`) su ogni operatore, incluse le coppie che si toccano.

Ciò che il frontend *non* contiene è la semantica degli operatori: le condizioni SQL, le soglie ammesse da ciascun operatore e le loro letture in parole sono servite dal backend attraverso `/api/iseql/vocabulary`, e il frontend le usa come dati. La Sezione 5.3 motiva questa scelta.

I componenti di interfaccia (finestre modali, caselle di scelta, schede) sono di **bits-ui**, lo stile è **Tailwind CSS**, entrambi già in uso in WATCHOUT: l'editor è un pannello dell'applicazione e ne segue le convenzioni.

4.4 Livello percettivo: modelli e librerie

Il livello percettivo è di WATCHOUT ed è descritto qui perché determina lo schema e il vocabolario su cui il compilatore lavora.

Sul lato visivo i fotogrammi, campionati con OpenCV, sono inviati a un modello visione-linguaggio attraverso gli SDK dei fornitori (`google-genai`, `mistralai`; il sistema supporta anche l'esecuzione locale). La risposta, in JSON, elenca oggetti con classe e identificatore e relazioni tra oggetti tratte da un vocabolario chiuso: `carrying`, `running`, `enter_or_exit_vehicle`, `suspicious_near_vehicle`, `physical_altercation`, e le altre che compaiono come valori di `RelationType`. La persistenza degli identificatori tra fotogrammi è affidata a una memoria di *embedding*: ogni oggetto rilevato è codificato con SigLIP [18] e cercato per somiglianza in **ChromaDB**; se esiste un vettore abbastanza vicino, l'oggetto riceve l'identificatore già assegnato. Nessun addestramento è coinvolto. Per le query questo garantisce che `ClassID` sia confrontabile tra intervalli diversi, cioè che le condizioni incrociate abbiano senso.

Sul lato sonoro, `ffmpeg` estrae la traccia audio a 16 kHz mono; **PANNs** CNN14 [6], eseguito su CPU, la classifica in finestre sulle 527 classi di AudioSet, mappate su nove classi di interesse; in alternativa **Qwen2-Audio-7B-Instruct** [15], quantizzato a 4 bit su GPU, risponde a una domanda sulla finestra. Le finestre consecutive con la stessa classe sono fuse in un intervallo scritto in `AudioPerInterval` con la sua confidenza.

Le due pipeline terminano nello stesso database con lo stesso schema concettuale (predicato, partecipanti, intervallo di fotogrammi). Un solo compilatore basta quindi per gli eventi visivi, sonori e multimodali, e un cambio di modello percettivo non tocca le definizioni degli eventi.

Capitolo 5

Architettura del sistema

Questo capitolo descrive il sistema realizzato: come un evento disegnato diventa una riga del registro degli eventi, come quella riga diventa SQL al momento della rilevazione, e quali decisioni hanno determinato la forma di ciascun passaggio. L'esposizione segue il flusso dei dati, non la cronologia del lavoro; dove una decisione è stata presa, cambiata e ripresa, si riporta l'esito e la ragione, e il diario di progettazione tenuto durante lo sviluppo resta il riferimento per il percorso.

5.1 Vista d'insieme

La Figura 5.1 mostra i componenti. Il livello percettivo di WATCHOUT (Sezione 4.4) scrive intervalli visivi e sonori nel database. L'**editor** nel browser produce un **modello di evento**, un documento JSON che descrive intervalli, operatori temporali, condizioni e proiezione. Il **salvataggio** scrive il modello nel **registro** degli eventi, la tabella `EventSpec`, accanto agli eventi predefiniti. Al momento della **rilevazione**, il servizio di analisi legge il modello dal registro, lo **compila** in SQL con le soglie fornite dall'interfaccia, ed esegue la query sul database dell'analisi; le righe restituite sono le occorrenze dell'evento, mostrate nell'interfaccia con gli intervalli e i fotogrammi che le sostengono.

Due proprietà dell'architettura vanno dette subito perché tutto il resto le serve.

La prima è che **ogni evento passa dalla stessa strada**. I sedici eventi predefiniti non sono più query scritte a mano nel codice: sono modelli nel registro, compilati al momento della rilevazione esattamente come quelli definiti dall'utente. Non era così all'inizio del lavoro, quando il compilatore affiancava le query esistenti e la rilevazione ricadeva su di esso solo per i nomi che il catalogo non conosceva; il passaggio a "tutto compila da un modello", deciso da Rifqialdi dopo che l'equivalen-

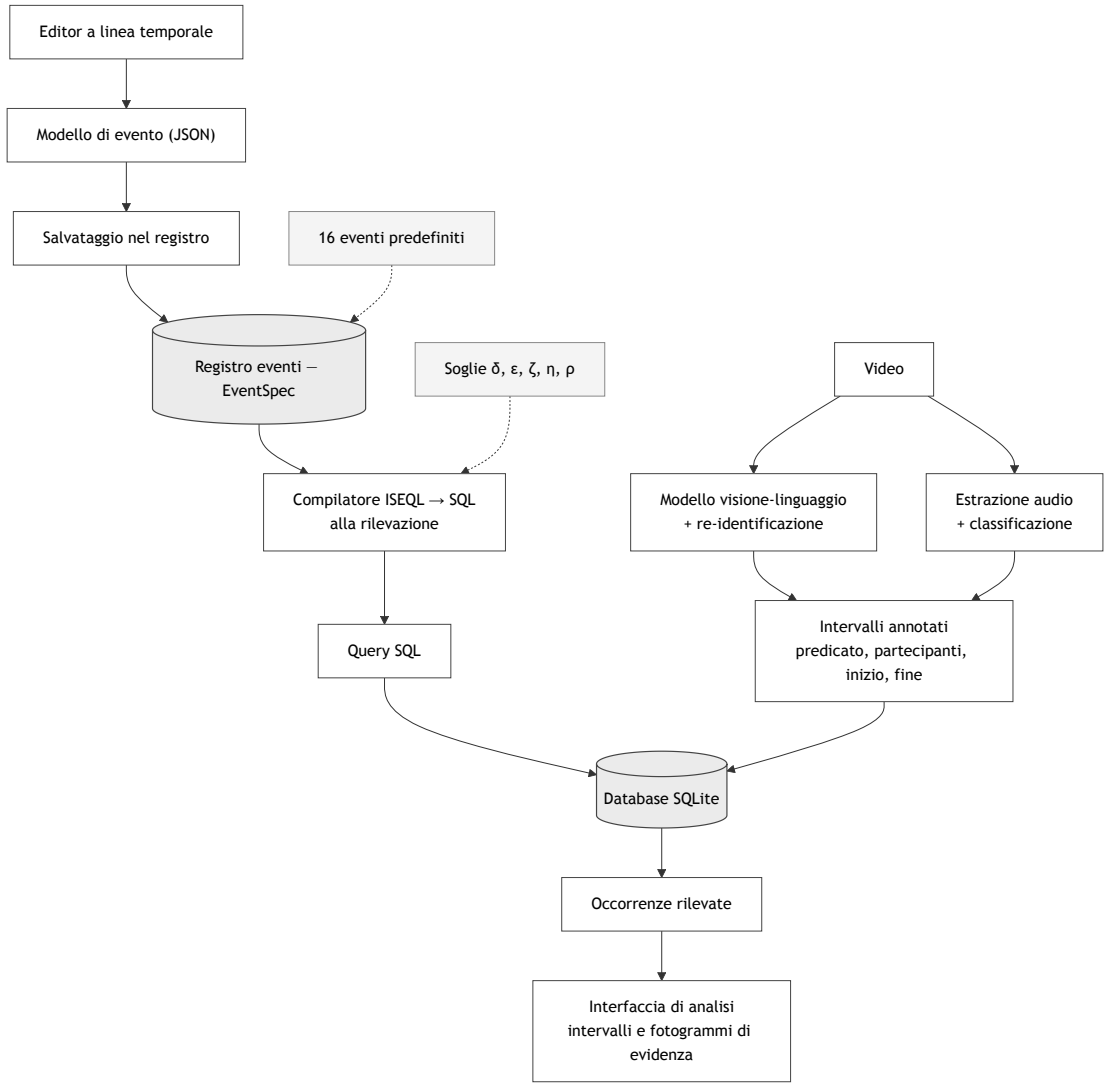


Figura 5.1: L'architettura del sistema integrato: l'editor produce un modello, il salvataggio lo scrive nel registro, la rilevazione lo compila in SQL ed esegue la query sugli intervalli prodotti dal livello percettivo.

za tra le due strade era stata dimostrata, ha eliminato la seconda implementazione. Le query scritte a mano sopravvivono nei test, come oracolo.

La seconda è che la **compilazione avviene alla rilevazione**, non al salvataggio (*compile-at-detect*). Il registro conserva il modello, non il SQL. Le soglie temporali sono parametri con nome, risolti con i valori che l'interfaccia passa a ogni richiesta di rilevazione: cambiare "entro due secondi" in "entro cinque" non richiede di modificare l'evento, e lo stesso modello produce query diverse per analisi diverse, perché l'identificatore dell'analisi è parte della compilazione.

5.2 Il ponte tra i due modelli dei dati

5.2.1 Ricavare la posizione dal tipo

Il Capitolo 3 ha lasciato aperto il primo problema: il Builder ragiona su argomenti in posizione, arg_1 e arg_2, mentre WATCHOUT conserva i partecipanti di un intervallo come insieme non ordinato in `VisualParticipant`, con una classe e un identificatore ma senza colonna di posizione. Un'espressione come $\sigma_{M_1.arg_1=M_2.arg_1}$ non ha significato finché non si dice cosa sia arg_1.

La soluzione adottata è che *la posizione si ricava dal tipo*. Ogni predicato ha una firma, registrata nel vocabolario delle relazioni di WATCHOUT, che dice quali classi di partecipanti prende e in che ordine: *carrying* è (*PersonID*, *ObjectID*), *enter_or_exit_vehicle* è (*PersonID*, *VehicleID*), *running* è (*PersonID*). Quell'informazione esisteva già nel codice, usata per costruire i prompt del modello visione-linguaggio; l'integrazione la usa come sistema di tipi. La Tabella 5.1 mostra la corrispondenza per i predicati coinvolti negli eventi.

Predicato	Firma	arg_1, arg_2
`running`	(PersonID)	person
`gunshot_visible`	(PersonID)	person
`carrying`	(PersonID, ObjectID)	person, object
`enter_or_exit_vehicle`	(PersonID, VehicleID)	person, vehicle
`suspicious_near_vehicle`	(PersonID, VehicleID)	person, vehicle
`vehicle_collision`	(VehicleID)	vehicle
`explosion_visible`	(VehicleID $\vee$ ObjectID)	vehicle oppure object
`physical_altercation`	(PersonID, PersonID)	person, person
classi sonore	(nessun partecipante)	

Tabella 5.1: Le firme dei predicati e la classe che ciascuna posizione seleziona.

Con la firma, arg_i di un intervallo è "il partecipante della classe che la firma mette in posizione i", e il compilatore lo ottiene con un join sulla tabella dei partecipanti filtrato per classe, uno per posizione:

```
JOIN VisualParticipant VP_1
  ON VPI.RelationID = VP_1.RelationID AND VP_1.Class = 'person'
JOIN VisualParticipant VP_2
  ON VPI.RelationID = VP_2.RelationID AND VP_2.Class = 'vehicle'
```

La stessa corrispondenza era stata realizzata, in una prima versione del lavoro, come vista SQL: `EventInterval` presenta ogni intervallo visivo e sonoro in una riga con colonne tipizzate (`person_a`, `person_b`, `vehicle`, `object`) e attributi derivati come `n_persons`.[1] Le due forme dicono la stessa cosa, e la differenza è dove vive: nella definizione della vista, oppure nei join che il compilatore genera. La seconda ha prevalso perché il compilatore deve comunque conoscere la firma del predicato per sapere quante posizioni risolvere, e con i join la conoscenza sta tutta in un posto.

Un caso è ambiguo per costruzione: *physical_altercation* ha due partecipanti della *stessa* classe, e la classe non basta a distinguere le posizioni. Nei dati non è raro: nel database dell'ablazione visiva con re-identificazione otto dei dieci intervalli di *physical_altercation* hanno più di una persona. La convenzione adottata, già usata dalla query originale di *fight*, è che la prima posizione sia l'identificatore minore e la seconda il maggiore: arbitraria ma deterministica, e senza contenuto semantico perché la relazione è simmetrica. Se un giorno esistesse una relazione asimmetrica tra due partecipanti della stessa classe, servirebbe una colonna di posizione vera, e questo è annotato come limite.

5.2.2 Lo scoping per analisi

Un database contiene molte analisi, ciascuna con il proprio `AnalysisID`, e una query deve confrontare intervalli della stessa analisi. Nel codice originale il filtro sull'analisi era iniettato dal chiamante nelle clausole `WHERE`, e l'identificatore aveva come default la stringa vuota, nel qual caso nessun filtro veniva applicato. Eseguendo *vehicle_escape* senza identificatore sul database dell'ablazione, la query restituiva **74 righe invece di 6**: corse e ingressi in veicolo di analisi diverse accoppiati tra loro. I risultati pubblicati non ne risentivano, perché la valutazione passava sempre l'identificatore, ma la correttezza dipendeva da una convenzione.

[1] La vista esiste ancora nello schema e la usa un secondo compilatore, rimasto come riserva per i modelli conservati come file anziché nel registro. Le due forme sono equivalenti sui dati; il resto del capitolo descrive solo il percorso del registro, che è quello che un evento disegnato segue.

La soluzione è duplice e viene da entrambi gli autori. L'autore di WATCHOUT ha reso l'identificatore dell'analisi obbligatorio e posizionale in tutte le funzioni di rilevazione, senza default; il compilatore, per parte sua, lo scrive dentro ogni CTE (`AND AnalysisID =` *id*), così una query compilata è corretta per costruzione anche se chi la esegue dimentica il filtro. L'identificatore è interpolato nel testo della query anziché legato come parametro: è una scelta accettabile qui, perché proviene dal registro delle analisi e non dall'utente, ma un sistema aperto a identificatori arbitrari dovrebbe legarlo.

5.3 Il modello di evento

Il modello è il formato in cui un evento esiste tra l'editor e il compilatore. È il JSON del Builder, esteso: un file scritto dall'applicazione originale si carica ancora, e i campi aggiunti sono facoltativi. Il modello di *vehicle_escape* nel registro, abbreviato, è:

```
{ "delta_unit": "seconds",
  "intervals": [
    { "pred": { "name": "running", "arguments": ["person"] },
      "ts": 0, "te": 100 },
    { "pred": { "name": "enter_or_exit_vehicle",
                "arguments": ["person", "vehicle"] },
      "ts": 50, "te": 150 } ],
  "operator_overrides": [ { "pair_idx": 1, "operator": "Bef" } ],
  "delta_map": { "0": { "delta": "delta_visual_vehicle_escape",
                        "zeta": "zeta_visual_vehicle_escape",
                        "rho": "rho_visual_vehicle_escape" } },
  "cross_conditions": [ { "left_alias": "M1", "left_attr": "arg1", "op": "=",
                          "right_alias": "M2", "right_attr": "arg1" } ],
  "custom_projection": [ "M1.arg1", "M2.arg2", "M1.st", "M2.et" ],
  "iseql_text": "..." }
```

Ogni campo corrisponde a un costrutto del Capitolo 3. Il campo `intervals` contiene gli alias $M_1, M_2, \ldots$ con predicato, classi degli argomenti e coordinate disegnate; le condizioni incrociate stanno in `cross_conditions` e la proiezione in `custom_projection`; per gli eventi multimodali, un operatore insiemistico con le proiezioni delle due metà. Un predicato può avere nomi alternativi (`alt_names`) per la disgiunzione, e una condizione può vincolare un attributo di un solo intervallo (condizione di tipo `attribute`): sono due dei quattro costrutti che il Builder non aveva.

Due campi contengono le decisioni di progetto più importanti del capitolo.

L'operatore è memorizzato, non ricalcolato. `operator_overrides` registra l'operatore scelto per ogni coppia adiacente. Il Builder lo ricalcolava a ogni apertura dalla posizione delle barre, e finché il modello serviva solo a generare testo la differenza non contava. Una query eseguita, invece, deve dipendere da ciò che l'autore ha *deciso*, non da dove ha lasciato due rettangoli: se l'operatore fosse ricavato dal disegno, spostare una barra di un pixel per motivi estetici cambierebbe il significato dell'evento. La geometria resta utile, e l'editor la usa per *proporre* l'operatore; ma ciò che si salva è la scelta.

Le soglie sono nomi, non numeri. `delta_map` associa a ogni coppia i *nomi* dei parametri δ, ζ, ρ (ed ε, η dove l'operatore li prende), non i valori. È ciò che rende una soglia raggiungibile: il servizio raccoglie i nomi richiesti da un evento, l'interfaccia mostra un controllo per ciascuno, e il valore arriva al momento della rilevazione. Un numero scritto nel modello compilerebbe, ma nessuno potrebbe regolarlo. Il campo `delta_unit` dice se i valori sono in secondi o in fotogrammi; in secondi, il compilatore divide gli estremi per la frequenza dell'analisi (`CAST(StartFrame AS REAL)/30`) così che soglie e attributi di durata restino confrontabili tra video a frequenze diverse.

Il modello ha due definizioni, le classi Python del backend e i tipi TypeScript del frontend (Sezione 4.3), che devono coincidere. Ciò che invece esiste *una volta sola* è la semantica: le condizioni SQL degli operatori, quali soglie ciascuno accetta e come ciascuno si legge in parole stanno in `iseql/helpers.py`, e il frontend le riceve da `/api/iseql/vocabulary`. Una seconda copia in JavaScript, anche corretta oggi, sarebbe una seconda cosa da tenere allineata, ed è esattamente il tipo di duplicazione che la Sezione 5.4.2 racconta di aver dovuto rimuovere.

5.4 Il compilatore

5.4.1 Da modello a SQL

Il compilatore (`iseql/compiler.py`) trasforma un modello in una query SQL sulle tabelle degli intervalli. La traduzione segue la struttura del modello: una CTE per alias, la condizione di ciascun operatore nella `WHERE` insieme alle condizioni incrociate, e la proiezione nella `SELECT`. Gli alias compaiono tutti nella clausola `FROM`, quindi formalmente il prodotto è cartesiano e sono le condizioni a ridurlo; è la forma che SQLite valuta comunque per un join su disuguaglianze, e rende la query più facile da leggere accanto all'espressione ISEQL da cui viene. Per il modello di *vehicle_escape* sopra, con $\delta = 2$ secondi e $\zeta = \leq$, il SQL prodotto è:

```
WITH M1 AS (
  SELECT VP_1.ClassID AS arg1,
         CAST(VPI.StartFrame AS REAL)/30 AS st,
         CAST(VPI.EndFrame AS REAL)/30 AS et
  FROM VisualPerInterval VPI
  JOIN VisualParticipant VP_1
    ON VPI.RelationID = VP_1.RelationID AND VP_1.Class = 'person'
  WHERE RelationType = 'running'
    AND AnalysisID = 'gemini_3_6_flash_s1'
),
M2 AS (
  SELECT VP_1.ClassID AS arg1, VP_2.ClassID AS arg2,
         CAST(VPI.StartFrame AS REAL)/30 AS st,
         CAST(VPI.EndFrame AS REAL)/30 AS et
  FROM VisualPerInterval VPI
  JOIN VisualParticipant VP_1
    ON VPI.RelationID = VP_1.RelationID AND VP_1.Class = 'person'
  JOIN VisualParticipant VP_2
    ON VPI.RelationID = VP_2.RelationID AND VP_2.Class = 'vehicle'
  WHERE RelationType = 'enter_or_exit_vehicle'
    AND AnalysisID = 'gemini_3_6_flash_s1'
)
SELECT M1.arg1, M2.arg2, M1.st, M2.et
FROM M1, M2
WHERE (M1.et <= (M2.st + 0) AND (M2.st - M1.et) <= 2)
  AND (M1.arg1 = M2.arg1)
```

Si riconoscono, in ordine: la selezione sul predicato (`RelationType`), i join per posizione della Sezione 5.2.1, lo scoping per analisi, la conversione in secondi, e nella `WHERE` la condizione di *Before* dell'equazione (3.3) con $\rho = 0$ seguita dalla condizione incrociata. Un predicato sonoro produce una CTE su `AudioPerInterval` invece che sulle tabelle visive, senza join di partecipanti; un evento multimodale produce due query di questo tipo unite da `UNION`. La decisione se un predicato sia sonoro non è cablata nel compilatore ma letta dalla tassonomia audio configurata, che il chiamante deve fornire: il compilatore rifiuta di indovinare.

5.4.2 Una sola definizione per operatore

La condizione nella `WHERE` non è scritta dal compilatore. È restituita dalla funzione `iseql_before` di `helpers.py`, la stessa che le query originali di WATCHOUT usavano, e lo stesso vale per gli altri sette operatori: il compilatore cerca la funzione dell'operatore nella tabella `OPERATOR_HELPERS`, la chiama con gli alias e le soglie, e inserisce il testo che riceve.

Non è stato così dall'inizio, e vale la pena dire perché. Quando il compilatore fu scritto, i moduli di WATCHOUT contenevano quattro operatori, di cui una sovrapposizione simmetrica e senza parametri; il Builder ne esprimeva otto, con δ ed ε. Il compilatore li implementò tutti in proprio, perché non c'era nulla da chiamare. Cinque commit dopo, WATCHOUT implementò gli otto operatori secondo le definizioni pubblicate, e da quel momento esistevano **due implementazioni della stessa algebra**: nessuna delle due errata, e nessun test le confrontava, perché il compilatore non importava l'altra.

Il costo si è visto prima che la duplicazione fosse rimossa. Il modello multimodale di *vehicle_collision* usava Bef $\vee$ Ovl mentre la query originale usava SP, e tutti i test passavano, perché la condizione C era confrontata come "ha rilevato qualcosa" da entrambe le parti. Sull'analisi `s15` il modello restituiva una quarta occorrenza, l'intervallo $(60, 210)$, che la query non restituiva. Il test di equivalenza confronta ora gli *insiemi* di occorrenze, e da quando il compilatore delega ai moduli di WATCHOUT il caso non può ripresentarsi: c'è una definizione per operatore, e chi la cambia cambia entrambe le strade.

Delegare ha richiesto di accettare le definizioni di WATCHOUT dove le due differivano. La strettezza di *Before* è stata la prima: WATCHOUT usa $\zeta = <$, il Builder $\leq$, e i due disaccordano solo sulle coppie che si toccano; la scelta è caduta sulla lettura stretta come valore predefinito, perché su fotogrammi a estremi inclusi il fotogramma condiviso è simultaneità (Sezione 3.1), e il rilevatore dell'editor è stato allineato nello stesso senso, perché un'interfaccia che etichetta "prima" una coppia che la query tratta come sovrapposta mentirebbe. *Left Overlap* è stata la seconda: la definizione di WATCHOUT ammetteva coppie disgiunte, e l'autore l'ha ristretta alla sovrapposizione dopo la discussione. Tutto questo è stato possibile perché ISEQL+ rende la strettezza un parametro esplicito: due persone possono disaccordare su ζ e vederlo scritto.

5.4.3 Cosa il compilatore rifiuta

Il compilatore non si limita a tradurre: rifiuta i modelli che non descrivono una query sensata, e il rifiuto è un errore con un messaggio, non una query vuota. Le verifiche riguardano la struttura del modello (un intervallo senza predicato, estremi non interi, una condizione incrociata senza uno dei suoi termini, un'operazione insiemistica con un solo operando) e la coerenza dell'espressione. Tre di queste ultime meritano una parola.

Un intervallo non può **mescolare predicati visivi e sonori** tra le sue alternative: le due modalità stanno in tabelle diverse, quindi un intervallo che sia "*running* oppure *shout*" non ha una CTE da cui nascere. La decisione su quale sia l'una e quale l'altra non è cablata nel compilatore ma letta dalla tassonomia audio configurata, che il chiamante deve fornire: se non la fornisce, il compilatore rifiuta di indovinare.

Una query non può **mescolare il dominio dei fotogrammi e quello dei secondi**: un modello che proietta `M1.sf` e vincola `M2.st` confronterebbe numeri che non hanno la stessa unità. L'errore lo dice e chiede di sceglierne uno.

Infine, se due intervalli adiacenti sono disposti in un modo che il rilevatore non sa classificare, il compilatore non sceglie un operatore ragionevole al posto dell'autore: chiede che l'operatore sia dichiarato esplicitamente, nominando la coppia.

Il criterio comune è quello che attraversa tutto il capitolo: un modello ambiguo o incoerente deve fermarsi al compilatore con un messaggio, perché il modo di fallire che l'integrazione esiste per rimuovere è la rilevazione che restituisce zero righe senza che nessuno sappia perché. Quando compila per l'editor, il compilatore controlla anche che ogni predicato esista nel vocabolario configurato e che i suoi argomenti rispettino la firma. Alla rilevazione questo controllo non viene ripetuto: un modello scritto nel registro senza passare dall'editor, che nomina un predicato mai prodotto dalla percezione, compila e non trova nulla, il che è indistinguibile da un evento che non si è verificato. La Sezione 7.3 lo elenca tra le cose da chiudere.

5.4.4 Le soglie

Resta la terza distanza elencata nella Sezione 3.3: nel Builder la soglia era la distanza disegnata. L'obiezione, sollevata da Rifqialdi, era che ISEQL non si occupa di quanto siano lunghi o distanti gli intervalli, ma di δ, ε, ζ, η, ρ; un editor che compila il divario tra due rettangoli come vincolo risponde a una domanda che il linguaggio non pone. L'obiezione era giusta a metà: le posizioni assolute non raggiungevano mai il SQL, ma δ sì, calcolato come $s.T_s - r.T_e$ sul disegno.

La regola adottata è che **un valore disegnato è una proposta, non una decisione**. Il rilevatore ricava dalle coordinate un valore per δ o ε, e quel valore è mostrato, ma ciò che finisce nel modello è quello che l'autore scrive nella finestra dell'operatore: un numero, il nome di un parametro da regolare in seguito, oppure ∞ per nessun limite. La distinzione conta perché la geometria della linea temporale non porta informazione sulla soglia: due barre distanti dieci unità di disegno non dicono "entro dieci secondi", dicono soltanto "poi".

La stessa regola vale a valle. Le soglie non hanno valori di default nel backend, e il servizio di rilevazione rifiuta una richiesta che ne ometta una richiesta dall'evento: un'omissione produce un errore che nomina il parametro mancante, non una query eseguita con un numero scelto da qualcun altro.

Con questo, ∞ è diventato dicibile ovunque: `inf`, ∞ e `math.inf` sono accettati come grafie della stessa cosa, e *Before* senza soglia compila nella forma senza vincolo numerico, come il passaggio di oggetto della Sezione 3.4 richiede.

5.5 Registro e servizio

5.5.1 Il salvataggio nel registro

Il registro degli eventi è la tabella `EventSpec`: una riga per evento, con la condizione (A, B o C), il modello JSON e le date di creazione e modifica. I sedici eventi predefiniti sono righe di questa tabella; un evento definito dall'utente è una riga uguale alle altre, e nulla nel servizio di analisi distingue le due origini.

Il pulsante *Save event* dell'editor compie il passaggio dal disegno alla riga. Se l'evento ha un identificatore nuovo, il modello è inviato con `POST /api/events`; se l'identificatore esiste già per quella condizione, con `PUT`, e l'evento viene sostituito. Il corpo della richiesta è il modello nella forma della Sezione 5.3, con la condizione e il testo ISEQL renderizzato accanto.

Il modello è **verificato prima di essere usato**, ma non dal registro: la riga viene scritta come arriva. A verificarla è l'editor, che tiene il testo ISEQL e il SQL allineati al modello a ogni modifica chiedendoli al server, quindi un modello che il compilatore rifiuta lo dichiara mentre lo si costruisce e non alla prima rilevazione. È una garanzia dell'interfaccia e non del servizio, e la Sezione 7.3 la elenca tra le cose da rendere strutturali.

5.5.2 La rilevazione

Il servizio di analisi ha un solo punto d'ingresso per la rilevazione, l'endpoint `detect` di un'analisi, che riceve il nome dell'evento, la condizione e le soglie. Il servizio legge dal registro le righe della condizione e per ciascuna chiama il compilatore con il modello, le soglie ricevute, l'identificatore dell'analisi, la frequenza dei fotogrammi e l'insieme dei predicati sonori della tassonomia configurata. Il SQL prodotto è eseguito con `run_sql_detection`, e le righe restituite sono le occorrenze.

Le soglie sono obbligatorie: `_require_deltas` rifiuta una richiesta che ometta un parametro usato da qualunque evento della condizione, e il backend non conserva valori di default. Chi li fornisce è l'interfaccia, che li ottiene da `default_deltas_for`: per ogni evento, i nomi in `delta_map` accoppiati ai letterali nel testo ISEQL. È così che un evento pubblicato compare nell'interfaccia di analisi con un controllo per ogni sua soglia, con il nome che l'autore le ha dato e il valore che ha deciso, modificabile a ogni rilevazione senza toccare l'evento.

Il rifiuto è sempre esplicito. Un evento che il registro non conosce, una soglia mancante, un predicato che la tassonomia non contiene, producono un errore che dice cosa manca e cosa esiste; nessun percorso restituisce un insieme vuoto per un errore di configurazione. È una scelta uniforme in tutto il sistema, dal controllo dei tipi del compilatore alla validazione di FastAPI, e discende dal requisito forense: un risultato vuoto deve significare che l'evento non è avvenuto, non che qualcosa non era configurato.

5.5.3 Il ciclo, dimostrato

La proprietà per cui l'integrazione esiste si verifica con un evento che non esiste da nessuna parte nel codice. *panic_flight*, definito come un'esplosione visibile sovrapposta a una corsa, è stato inviato all'API; è stato compilato e salvato; è comparso nell'elenco degli eventi della condizione A accanto ai sei predefiniti; e una richiesta di rilevazione per `panic_flight` sulla scena 10 ha restituito sette occorrenze tra i fotogrammi 24 e 240, dal secondo 1 al secondo 10 del video. Nessun file Python è stato modificato e il backend non è stato riavviato. La Sezione 6.1 ripete la dimostrazione dall'interfaccia, con un evento disegnato.

5.6 L'editor

L'editor è la scheda *Events (ISEQL)* della pagina di configurazione degli eventi. È organizzato come il Builder originale, perché quella era l'interfaccia che le persone

sapevano usare: la linea temporale al centro e i pannelli *Intervals*, *Sets* e *Set Expression* accanto. Tutto il resto si apre da ciò che si vede: un clic su una barra apre la finestra di quell'intervallo, uno sul distintivo tra due barre la finestra dell'operatore, uno sul nome di un insieme la finestra che contiene proiezione e condizioni. L'espressione ISEQL e il SQL generato si leggono nella vista testuale, un editor dell'espressione con completamento che lavora sullo stesso modello; non è descritto qui.

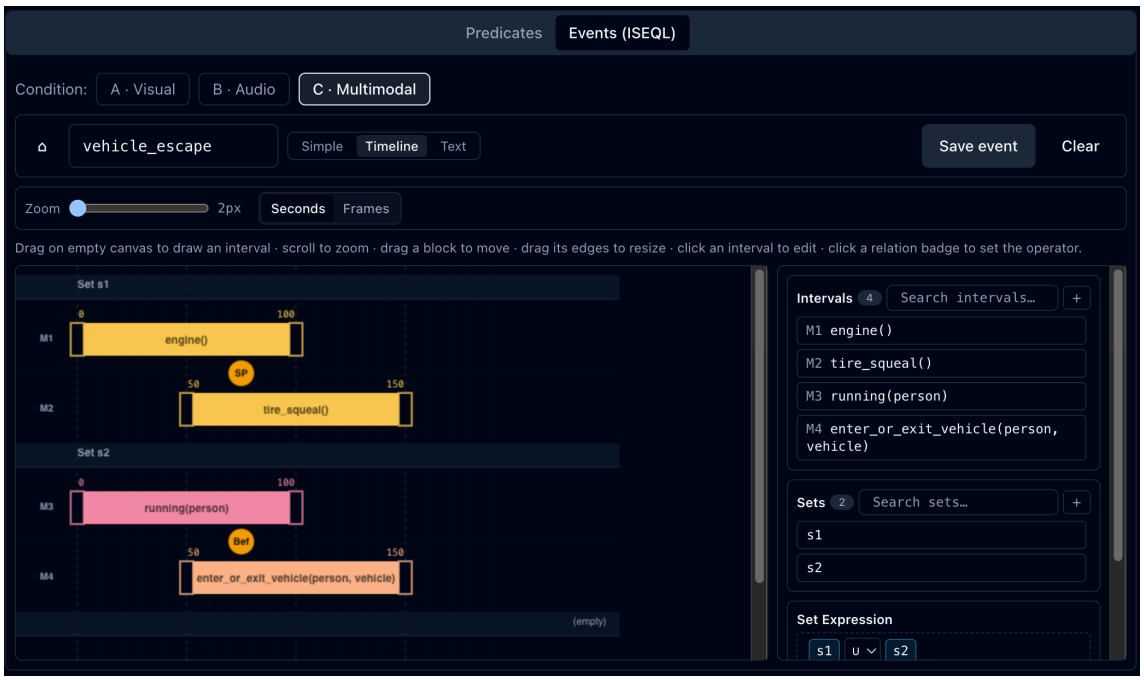


Figura 5.2: L'editor con un evento multimodale: l'insieme `s1` contiene i due intervalli sonori (*engine*, *tire_squeal*) legati da SP, l'insieme `s2` i due visivi (*running*, *enter_or_exit_vehicle*) legati da Bef, e l'espressione insiemistica in basso a destra li unisce. I distintivi circolari tra le barre mostrano l'operatore di ciascuna coppia.

5.6.1 La linea temporale

Ogni intervallo del modello è una barra su una linea temporale astratta, con il nome del predicato e le classi degli argomenti; le barre si trascinano, si allungano, si aggiungono da un elenco di predicati (il vocabolario servito dal backend, filtrato per condizione) e si rimuovono. Tra due barre adiacenti un distintivo mostra l'operatore.

Il distintivo mostra l'operatore *memorizzato* se l'autore ne ha scelto uno, altrimenti quello rilevato dalla geometria. Il rilevatore lato client è una trascrizione riga per riga di quello del backend, e serve solo a etichettare il disegno mentre si trascina; ogni volta che il modello cambia, il testo ISEQL e il SQL mostrati vengono dal server

(`/api/iseql/model/compile`), che risolve gli operatori con la definizione unica. La pagina non decide nulla che il backend non confermi.

Due convenzioni governano il disegno, ed entrambe derivano dalla Sezione 5.4.4.

Il disegno dice l'operatore, il testo dice le soglie. Spostare una barra fino a sovrapporla alla vicina cambia l'operatore da *Before* a *Left Overlap*: è il fatto qualitativo che ISEQL legge da due intervalli. La soglia δ non si muove. L'editor riporta il valore "implicito" dalle coordinate, ma non lo applica: δ:10 resta quello che la query dice finché l'autore non scrive un altro numero. Un trascinamento che cambia l'operatore elimina l'eventuale operatore memorizzato per quella coppia, perché descriverebbe una disposizione che non esiste più, e un test lo afferma.

L'authoring è inclusivo, la misura è stretta. Due barre disposte una di seguito all'altra, con la fine della prima sull'inizio della seconda, significano "poi": il rilevatore le etichetta *Before*. Il SQL compilato per *Before* è stretto per impostazione predefinita, $\zeta = <$, perché opera su fotogrammi misurati, dove un fotogramma condiviso è simultaneità; un evento che vuole la lettura inclusiva la dichiara con $\zeta = \leq$, come fanno gli eventi del registro (Sezione 3.4). Le due convenzioni non sono in conflitto perché riguardano oggetti diversi, coordinate disegnate e intervalli misurati, e si incontrano nel compilatore. Il punto è stato chiarito nel modo più costoso: due rilevatori, uno per l'editor e uno per il motore, disaccordavano proprio su questa coppia, $(0, 50)$ e $(50, 100)$, e una prima correzione andò nel verso sbagliato, rendendo stretto l'editor; ne seguì che il passaggio di oggetto della scena 21, con i due trasporti registrati come $[96, 120]$ e $[144, 168]$, non veniva più rilevato. Da allora la regola è una sola, il rilevatore del client ne è la trascrizione, e un test con due metà fissa entrambe le convenzioni.

Le modifiche strutturali, aggiungere, rimuovere o spostare un intervallo, richiedono attenzione perché tutto il resto del modello è indicizzato per posizione: le condizioni nominano M_3, gli operatori memorizzati portano l'indice della coppia, le soglie sono per coppia, la proiezione elenca $M_2.arg_1$. Rimuovere il secondo intervallo obbliga a rinumerare o eliminare tutti e quattro, e questa rinumerazione era il punto in cui il Builder originale si rompeva in silenzio: scambiando due intervalli, una condizione di durata scritta per una corsa finiva su un grido, il modello compilava, e la query significava altro. Le regole stanno in `model_edit.py`, che il backend espone come `/api/iseql/model/edit`: la rimozione di un intervallo elimina le condizioni e i campi della proiezione che lo nominano, rinumera quelli che nominano gli intervalli successivi, ed elimina gli operatori e le soglie delle due coppie in cui stava, perché la nuova coppia tra i suoi vicini è una relazione per cui nessuno ha ancora deciso nulla. L'editor attuale, però, non usa questo endpoint: tiene il modello nel browser,

e quando si cancella un intervallo lo toglie dalla lista senza applicare queste regole. Cancellando il secondo di tre intervalli, una condizione scritta per il secondo passa in silenzio al terzo, che ne prende l'alias; una condizione scritta per il terzo nomina un alias che non esiste più, e il compilatore la scarta senza segnalarlo; e l'operatore con le soglie che legava il primo al secondo resta sulla nuova coppia, tra il primo e quello che era il terzo. È lo stesso difetto del Builder originale, spostato dal server al browser, e il Capitolo 7 lo riporta tra i limiti.

Un modello può avere più **gruppi** di intervalli, ciascuno una linea temporale, combinati da un'espressione insiemistica ($\cup$, $\setminus$, $\cap$) modificabile in un editor ad albero: è così che si costruisce un evento multimodale, un gruppo per modalità.

5.6.2 Operatori e soglie

Il distintivo tra due barre apre la finestra *Temporal operator*, che è dove si decide la relazione tra quella coppia: l'operatore, scelto tra gli otto o lasciato al rilevamento automatico, e un controllo per ciascun parametro che quell'operatore prende, δ, ε, ζ, η, ρ, con ∞ disponibile per le soglie senza limite. Quali parametri abbiano senso dipende dall'operatore, *End Following* prende ε e mai δ, e questa informazione viene dal server insieme al vocabolario: una pagina che la stabilisse da sé starebbe decidendo cosa sono gli operatori, cioè l'errore appena descritto. I campi mostrano ciò che la query porta, non caselle vuote: il valore di base dell'operatore se il modello non dice nulla. Una soglia può essere un numero oppure un **nome**, ed è il nome a renderla regolabile dall'interfaccia di analisi; il nome è conservato accanto al suo default, così il testo ISEQL mostra δ:90 mentre il modello porta `delta_visual_chase`. Riaprire e salvare un evento non sostituisce i nomi con i numeri, cosa che in una versione precedente accadeva e rendeva l'evento non più regolabile.

I pannelli della **proiezione** e delle **condizioni incrociate**, che stanno nella finestra di un gruppo, sono stati costruiti dopo aver contato, sui sedici eventi del registro, quanto ciascun costrutto fosse usato: la proiezione da tutti e sedici, le condizioni incrociate da sei, le condizioni di durata da due, le condizioni di attributo contro un letterale da nessuno. La proiezione, che sembrava la meno urgente, era la meno facoltativa. Entrambi i pannelli offrono tutto e non fanno digitare nulla: gli alias, gli attributi che ciascun alias possiede secondo la firma del suo predicato, gli operatori di confronto sono menu, e un evento con condizioni e proiezione si costruisce senza scrivere un simbolo di ISEQL.

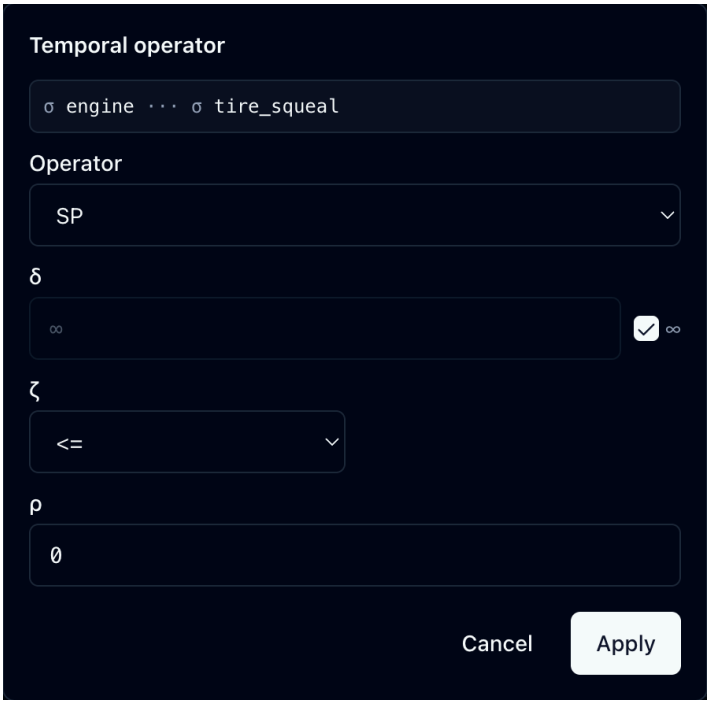


Figura 5.3: La finestra dell'operatore, aperta dal distintivo tra due barre: la coppia a cui si riferisce, l'operatore scelto fra gli otto, e un controllo per ciascun parametro che quell'operatore prende. Qui SP, che prende δ e non ε, con δ senza limite.

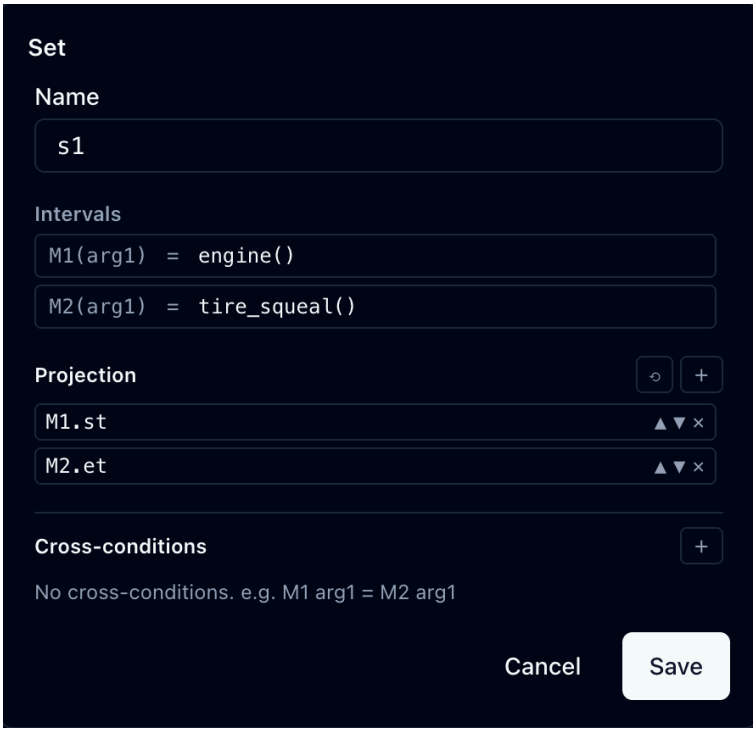


Figura 5.4: La finestra di un insieme: il nome, gli intervalli che contiene con le loro posizioni argomentali, la proiezione come lista ordinata di colonne, e le condizioni incrociate.

5.6.3 Salvare l'evento

Un identificatore e la condizione in cima all'editor, e il pulsante *Save event* scrive l'evento nel registro. Da quel momento compare nella pagina di analisi accanto agli eventi predefiniti, con le soglie che portano un nome tra i controlli regolabili a ogni rilevazione. Non c'è un passaggio di pubblicazione separato: l'evento è già stato compilato durante la costruzione, perché il server ricalcola il testo ISEQL e il SQL a ogni modifica, e la vista testuale li mostra.

5.7 Sviluppo in parallelo

WATCHOUT non è rimasto fermo durante questo lavoro: nello stesso periodo Ghiffary Rifqialdi ha continuato a svilupparlo nello stesso repository per la propria tesi [16], e le sue modifiche sono state integrate due volte. La seconda integrazione comprendeva quattordici commit e, tra questi, un editor grafico di eventi scritto in modo indipendente: la stessa funzionalità a cui lavorava questa tesi, al punto che entrambe le versioni avevano creato un file con lo stesso nome, `iseql-timeline.svelte`, assente nella base comune. Diciannove file erano in conflitto, e nessuno si poteva unire automaticamente.

Si è scelto di tenere l'editor descritto in questo capitolo e di accogliere tutto il resto, per la ragione che attraversa il capitolo. Poco prima, due implementazioni della rilevazione degli operatori erano state confrontate sui dati dell'ablazione e davano esiti diversi su **4050 coppie di intervalli** che condividevano un singolo fotogramma, lette dall'una come *Before* e dall'altra come sovrapposizione; ogni risultato *Before* sull'intero dataset era una di quelle coppie. Una terza implementazione indipendente del rilevatore, scritta in JavaScript, avrebbe ripetuto lo stesso errore in un altro linguaggio, il più lontano dai test. Il codice sostituito è stato eliminato e non lasciato irraggiungibile: una seconda fonte di verità resta un rischio anche quando nulla la chiama.

Le scelte sulla semantica del sistema (la strettezza di *Before*, la lettura di *Left Overlap*, l'obbligo di indicare l'identificatore di analisi e le soglie, la forma delle righe della condizione C) riguardavano definizioni che appartengono a WATCHOUT, e sono state concordate con Rifqialdi anziché fissate da una parte sola. Il risultato è che il sistema ne ha una versione unica. Alcune sono arrivate come codice nelle integrazioni successive; una, la restrizione di *Left Overlap* alla sovrapposizione, è arrivata nelle query compilate senza che il compilatore cambiasse di una riga, perché da quando il compilatore delega ai moduli di WATCHOUT le definizioni degli operatori sono le loro.

Capitolo 6

Funzionamento ed espressività

Il Capitolo 5 ha descritto il sistema componente per componente. Questo capitolo lo mostra in uso: il percorso di un evento dall'editor al risultato, ciò che il linguaggio realizzato permette e non permette di esprimere, il repository di video su cui il sistema è stato provato, e ciò che l'uso ha fatto emergere che la sola verifica dei risultati non aveva mostrato: prima i difetti trovati usandolo, poi l'opinione di un gruppo di utenti non esperti.

6.1 Un evento dall'editor al rilevamento

Si segue un evento non previsto dal sistema: una persona che si avvicina a un veicolo, vi resta accanto, e poi corre via. In ISEQL è un intervallo *suspicious_near_vehicle* seguito entro pochi secondi da un intervallo *running* della stessa persona:

```
π_{M1.arg1, M1.arg2, M1.st, M2.et} (
  σ_{M1.arg1 = M2.arg1} (
    σ_{pred="suspicious_near_vehicle"
        ∧ arg1="person" ∧ arg2="vehicle"}(M1)
    Bef(δ:3; ζ:<, ρ:0)
    σ_{pred="running" ∧ arg1="person"}(M2)))
```

Nessuna di queste righe viene scritta: il percorso che segue le produce.

Disegnare. Nella scheda *Events (ISEQL)*, con la condizione A selezionata, si aggiunge un intervallo scegliendo *suspicious_near_vehicle* dall'elenco dei predicati; la barra compare sulla linea temporale con le classi dei suoi argomenti, *person* e *vehicle*, prese dalla firma. Si aggiunge *running*: la seconda barra è collocata di seguito alla

prima, e il distintivo tra le due mostra *Bef*, perché due barre disposte una dopo l'altra significano "poi". Un clic sul distintivo apre la finestra dell'operatore, dove si conferma *Bef* e si trovano i controlli per δ, ζ e ρ.



Figura 6.1: Il disegno dell'evento: due intervalli sulla linea temporale e il distintivo *Bef* tra i due.

Vincolare. La finestra dell'insieme contiene i due pannelli che restano. In quello delle condizioni incrociate si compone $M_1.arg_1 = M_2.arg_1$ da tre menu: l'alias e l'attributo a sinistra, l'operatore, l'alias e l'attributo a destra; gli attributi offerti per ciascun alias sono quelli che la firma del suo predicato ammette. In quello della proiezione si scelgono le colonne del risultato: la persona e il veicolo dal primo intervallo, l'inizio del primo e la fine del secondo. A ogni modifica il server ricalcola il testo ISEQL e il SQL, che la vista testuale mostra aggiornati.

Salvare. Si dà all'evento un identificatore, si sceglie la condizione A, e si preme *Save event*. Il modello è scritto nel registro. Non c'è altro passaggio: il testo ISEQL e il SQL sono stati prodotti dal server a ogni modifica, quindi l'evento era già compilabile prima di essere salvato.

Rilevare. Nella pagina di analisi si riapre un'analisi già eseguita, qui quella della scena 19 del repository (Sezione 6.3), e si rilanciano le sole query, senza ripetere l'analisi del video. Il nuovo evento viene rilevato insieme ai sei predefiniti della condizione A e restituisce una riga, con le colonne scelte nella proiezione: la persona 2 e il veicolo 1, dal fotogramma 48, in cui comincia la sosta, al 216, in cui finisce la corsa. Nella stessa scena la persona 2 compare anche accanto a un secondo veicolo, ma in un intervallo che cade nello stesso fotogramma in cui comincia la corsa; con ζ uguale a $<$, *Before* chiede che la corsa cominci dopo la fine della sosta, e quella coppia non produce un'occorrenza.

La soglia scritta come numero resta fissata nel modello. Per renderla regolabile a ogni rilevazione le si dà un nome, scrivendo nella vista testuale

`Bef(δ:delta_visual_...)`: da quel momento compare tra i controlli della pagina di analisi come quelle degli eventi predefiniti (Sezione 5.4.4), e cambiarla non tocca l'evento, ma solo la query compilata per quella richiesta.

Il percorso ha prodotto un evento e lo ha reso rilevabile senza che nessuno scrivesse una riga di ISEQL o di SQL, senza modificare il codice e senza riavviare il backend. È la proprietà enunciata nell'introduzione. L'espressione ISEQL riportata in apertura è quella che l'editor ha renderizzato; il SQL che la pagina di analisi ha eseguito ha la forma della Sezione 5.4.

6.2 Espressività

6.2.1 I sedici eventi come modelli

Il criterio di espressività fissato nel Capitolo 3 era che i sedici eventi di WATCHOUT fossero esprimibili come modelli e che, compilati, producessero le stesse occorrenze delle query originali. La Tabella 6.1 elenca i sedici eventi con i costrutti che ciascuno richiede. Tutti sono nel registro come modelli, compilati alla rilevazione; le loro occorrenze coincidono con quelle delle query originali su ogni analisi dei tre database dell'ablazione, confrontate come insiemi di righe. Non è una verifica fatta una volta: è la forma in cui i sedici eventi esistono nel sistema.

Letta per colonne, la tabella dice qualcosa sull'uso reale del linguaggio. Nessun ramo ha più di due intervalli: gli eventi della condizione C arrivano a tre o quattro solo perché uniscono due rami. Di otto operatori temporali ne compaiono *due*, *Before* e *Start Preceding*, e sei eventi su sedici non ne usano nessuno, perché sono un singolo intervallo con una condizione. Le condizioni incrociate compaiono in sei eventi, la condizione di durata in due, la disgiunzione di predicati in tre; la proiezione è in tutti e sedici. Gli altri sei operatori sono disponibili e coperti dai test, ma nessun evento del catalogo ne ha avuto bisogno: la sequenza con soglia e la sovrapposizione all'inizio coprono ciò che la sorveglianza chiede a questi dati.

6.2.2 Cosa il sistema aggiunge

Rispetto al Builder originale, il sistema esprime i quattro costrutti elencati nella Sezione 3.3: la disgiunzione di predicati in un intervallo, la condizione su un attributo di un singolo intervallo (durata, confidenza), la condizione di conteggio dove il ponte la rende un attributo, e la disgiunzione tra operatori. Rispetto a WATCHOUT, aggiunge la possibilità di definire un evento senza scrivere SQL e di regolarne le

Evento	Cond.	Interv.	Costrutti
fight	A	1	condizione incrociata $arg_1 \neq arg_2$
suspicious_near_vehicle	A	1	condizione di durata (≥ 3 s)
gunshot_or_explosion	A	1	disgiunzione di predicati
vehicle_collision	A	1	selezione sul predicato
vehicle_escape	A	2	Bef_δ, condizione incrociata $=$
handoff	A	2	Bef_∞, condizioni incrociate $\neq$ e $=$
gunshot_or_explosion	B	1	selezione sul predicato sonoro
fight	B	2	Bef_δ tra predicati sonori
vehicle_escape	B	2	SP_∞
vehicle_collision	B	2	SP_∞, disgiunzione di predicati su entrambi
suspicious_near_vehicle	C	1	come in A: nessun correlato sonoro
handoff	C	2	come in A: nessun correlato sonoro
gunshot_or_explosion	C	2	$\cup$ delle due metà
fight	C	3	$\cup$ della metà visiva e di quella sonora
vehicle_collision	C	3	$\cup$, con SP nella metà sonora
vehicle_escape	C	4	espressione insiemistica, Bef e SP

Tabella 6.1: I sedici eventi del registro, con il numero di intervalli e i costrutti del linguaggio che ciascuno usa.

soglie per nome. Rispetto a entrambi, aggiunge che le soglie sono decise e non misurate, e che ogni operatore ha una definizione sola.

Alcune cose che si possono dire in ISEQL non hanno ancora una forma nell'editor. Gli operatori *Start Preceding* ed *End Following* non si disegnano, perché nessuna disposizione di due barre li implica: si scelgono dal menu dell'operatore, declassando una sovrapposizione. Il parametro di robustezza ρ e le strettezze ζ, η si impostano nel pannello delle soglie e non hanno una rappresentazione grafica, perché non descrivono una posizione ma una tolleranza.

6.2.3 Cosa non si può esprimere

Tre limiti sono stati riconosciuti e accettati.

Il modello è una **catena piatta**, non un albero: gli intervalli di un gruppo sono una lista ordinata e gli operatori legano coppie adiacenti. Un'espressione come (A Bef B) DJ C, in cui una sottoquery è argomento di un operatore, non ha forma. Sollevare il limite significherebbe sostituire la lista con un albero e ridisegnare la linea temporale per mostrarlo, e una linea temporale è per natura lineare. Nessuno dei sedici eventi lo richiede.

L'**aggregazione** non esiste come costrutto: non si può scrivere "almeno tre persone" se non per un attributo che il ponte precalcola, come `n_persons`. Il Builder non ha aggregati perché ISEQL non ne ha; la vista li nasconde per i casi previsti, e i casi non previsti restano fuori.

Il ponte tra i due modelli dei dati **collassa la molteplicità**: un intervallo con due persone e un veicolo dà una riga per posizione, con la convenzione minimo e massimo, dove la query originale avrebbe dato il prodotto cartesiano. Nei dati attuali il caso non si presenta, *suspicious_near_vehicle* ha al più una persona e un veicolo in tutti i suoi intervalli, ma è una differenza di significato e non un'ottimizzazione, ed è annotata come tale.

6.3 Il repository video

Il sistema è stato provato sul dataset costruito per la valutazione di WATCHOUT: **trenta scene** di sorveglianza, dieci secondi ciascuna, generate con Seedance 2.0 [17], un modello di generazione video da testo, a 1280×720 e 24 fotogrammi al secondo, da una telecamera fissa in posizione leggermente elevata, senza zoom né panoramiche. Le scene coprono sei tipi di evento, cinque scene per tipo, con variazioni di contesto (parcheggio, strada, ingresso di un edificio) per misurare la coerenza della rilevazione. Ogni scena è generata una sola volta da un prompt che descrive l'azione secondo per secondo, senza montaggio.

La verità di riferimento è annotata per fotogramma e per modalità. Per la scena 4, una rissa presso un muro, l'annotazione è:

Modalità	Classe	Inizio (s)	Fine (s)	Fotogramma	Fotogramma
audio	shout	0.000	1.500	0	36
audio	impact	1.500	1.667	36	40
visual	physical_altercation	1.583	5.625	38	135

Con questa annotazione ogni scena è etichettata, per ciascun tipo di evento e per ciascuna condizione, come positiva o negativa, e una condizione è valutata solo sulle scene in cui la sua modalità può percepire l'evento: il passaggio di oggetto e la sosta sospetta non hanno correlato sonoro e sono esclusi dalla condizione B, gli eventi fuori inquadratura sono esclusi dalla A. Le metriche sono precisione, richiamo e F1 per tipo di evento.

Il dataset non è opera di questa tesi. Ne ha fornito i **database dell'ablazione** su cui il sistema è stato provato. Le occorrenze dei modelli sono confrontate con quelle delle query di riferimento su tre di questi, uno visivo, uno sonoro e uno multimodale:

il database visivo prodotto da Gemini 3.6 Flash senza re-identificazione, con 200 intervalli su 29 analisi; quello sonoro, con 108 intervalli prodotti da Qwen2-Audio su 20 analisi, cioè le scene che hanno un correlato acustico; e quello multimodale, che affianca 103 intervalli visivi di Gemini 3.6 Flash con re-identificazione, su 30 analisi, a 64 intervalli sonori di Qwen2-Audio. Sono database registrati, non rigenerati a ogni esecuzione: ogni confronto è ripetibile e dà lo stesso risultato.

L'esecuzione completa della pipeline, dalla percezione alla rilevazione, è stata verificata nelle tre condizioni su un portatile senza GPU: condizione A con Gemini 3.6 Flash e poi con Ministral 3-14B sulla scena 10, condizione B con PANNs CNN14 sulla scena 10, condizione C con Ministral 3-14B e PANNs sulla scena 9. In tutti i casi l'evento atteso, *gunshot_or_explosion*, è stato rilevato; nella condizione C la rilevazione porta sia un identificatore visivo sia uno sonoro, cioè l'unione ha combinato evidenze che le due pipeline hanno trovato indipendentemente.

6.4 Ciò che l'uso ha fatto emergere

La verifica per confronto di risultati stabilisce che le occorrenze sono quelle giuste. Non stabilisce che siano *presentate* nel modo giusto, e la prima ora di uso del sistema attraverso l'interfaccia ha fatto emergere quattro difetti che nessun confronto di conteggi poteva vedere.

Le **colonne avevano il nome sbagliato**: gli eventi definiti da modello proiettavano `M1_sf` e `M1_ef` dove la tabella dei risultati si aspetta `StartFrame` ed `EndFrame`; le righe c'erano ed erano corrette, e la tabella le mostrava vuote.

Gli **intervalli si accoppiavano con se stessi**: *During* e le sovrapposizioni sono riflessive, una riga le soddisfa accoppiata a se stessa, e un modello a due intervalli inventava un evento da una sola occorrenza. Metà delle occorrenze del primo modello disegnato a mano erano di questo tipo. La query originale di *handoff* lo impediva con un confronto di identificatori; il compilatore lo fa ora per ogni coppia.

Nulla identificava gli intervalli dietro un'occorrenza: il compilatore non proiettava gli identificatori delle righe di partenza, in un sistema il cui argomento è la verificabilità. Ora proietta `M1_id`, `M2_id`, prefissati per modalità (`V38`, `S45`), ed è su questi che l'interfaccia risale ai fotogrammi di evidenza.

La **condizione C collassava le occorrenze**: sulla premessa, sbagliata, che le query multimodali originali restituissero una sola riga aggregata, i modelli della condizione C erano compilati con una proiezione di ampiezza, il minimo degli inizi e il massimo delle fini, e restituivano una riga sola dove le query originali ne restituivano una per occorrenza, fino a sei per *handoff*. La premessa era sbagliata perché `MIN(a,`

`b)` e `MAX(a, b)` in SQLite sono funzioni scalari (Sezione 4.1); il confronto booleano tra le due strade non poteva vederlo. La correzione, una riga nel servizio, è stata fatta in accordo con Rifqialdi, perché cambia ciò che ogni rilevazione multimodale mostra.

Quattro difetti nella prima ora sono una misura di quanto il sistema era stato usato, non di quanto fosse corretto: la verifica copre i percorsi che sono stati percorsi, e la Sezione 7.3 torna su ciò che resta da percorrere.

6.5 Soddisfazione degli utenti

La verifica per equivalenza stabilisce che un evento disegnato viene compilato nella query giusta. Non dice nulla su chi lo disegna. Eppure è da lì che la tesi parte: un evento si deve poter definire senza conoscere ISEQL né SQL, da parte di chi usa il sistema e non di chi lo ha scritto. Le due domande sono distinte anche nella letteratura sui sistemi rivolti all'utente: Albanese et al. [19], per esempio, affiancano alla misura dell'accuratezza una valutazione della soddisfazione degli utenti alle prese con compiti concreti. Questa sezione riporta una prima prova dello stesso tipo, più semplice, con il questionario NASA-TLX, e una valutazione euristica dell'interfaccia che ne precisa le indicazioni.

6.5.1 La prova con gli utenti

Come si è svolta. Il sistema è stato fatto provare a sette persone, da P1 a P7, tra i 19 e i 29 anni, che non conoscevano né ISEQL né WATCHOUT; sei di loro hanno poche o nessuna competenza informatica e non conoscono SQL. Dopo una breve spiegazione iniziale dell'editor a linea temporale, ciascuna lo ha usato liberamente, provando a costruire eventi, e al termine ha espresso le proprie opinioni su una scala, con il questionario NASA-TLX descritto più avanti. Non sono stati misurati tempi né esiti: la prova raccoglie il giudizio dei partecipanti, non le loro prestazioni.

Cosa è emerso. Il giudizio è stato concorde. Il sistema è risultato **efficace**, capace di fare ciò che promette, ma **poco pratico da usare**, per due ragioni precise. La prima riguarda il punto di partenza: il comando che crea un nuovo evento è un piccolo pulsante "+" accanto all'elenco degli eventi, senza etichetta, e trovarlo non è stato intuitivo. La seconda riguarda il lavoro sugli intervalli: al di fuori della spiegazione iniziale, l'interfaccia non accompagna chi li crea.

Il questionario NASA-TLX. Il **NASA-TLX**, il questionario sul carico di lavoro percepito usato anche da Albanese et al. [19], misura sei dimensioni da 0 a 100, dove un valore più basso è migliore. L'hanno compilato tutti e sette i partecipanti, dopo aver costruito un evento con la linea temporale. Era prevista anche una seconda attività, costruire lo stesso evento scrivendolo in ISEQL nella vista testuale, ma nessuno dei sette è riuscito a svolgerla: nessuno ha compreso la notazione abbastanza da cominciare. Tutti hanno compilato anche i quindici confronti a coppie della versione completa del questionario: in ogni confronto il partecipante sceglie quale delle due dimensioni ha contribuito di più alla fatica. Contando le scelte si ottiene per ciascuna dimensione un peso da 0 a 5, che dice quali criteri hanno richiesto lo sforzo maggiore, e il carico *pesato* è la media delle valutazioni con questi pesi. Di tutti sono noti età, competenze informatiche e conoscenza di SQL, e quindi dell'algebra relazionale su cui SQL si fonda. La Tabella 6.2 riporta i valori grezzi, la Tabella 6.3 gli stessi valori aggregati per profilo.

	Età	Informatica	SQL	M	F	T	P	S	R	Semplice	Pesato
P1	20	poche	no	30	50	45	30	30	15	33,3	38,3
			pesi	2	4	3	5	1	0		
P2	26	buone	sì	45	40	50	40	35	20	38,3	38,7
			pesi	2	2	4	2	2	3		
P3	19	nessuna	no	25	5	30	25	30	10	20,8	26,0
			pesi	3	1	4	4	3	0		
P4	22	nessuna	no	15	5	35	20	35	5	19,2	22,7
			pesi	4	1	5	3	1	1		
P5	21	nessuna	no	20	5	25	25	30	10	19,2	23,0
			pesi	4	1	5	3	2	0		
P6	23	nessuna	no	45	45	35	45	35	10	35,8	36,7
			pesi	5	2	0	3	2	3		
P7	29	nessuna	no	25	5	25	20	35	10	20,0	22,3
			pesi	4	2	2	3	3	1		

Tabella 6.2: Valori grezzi del NASA-TLX, da 0 a 100, dopo la costruzione di un evento con la linea temporale, con il profilo di ciascun partecipante: età, competenze informatiche e conoscenza di SQL. Le dimensioni sono (M) richiesta mentale, (F) fisica, (T) temporale, (P) prestazione, con 0 per pienamente riuscita, (S) sforzo, (R) frustrazione. Sono riportati anche i pesi ricavati dai confronti a coppie, da 0 a 5 con somma 15. Il carico semplice è la media delle sei valutazioni, quello pesato la somma delle valutazioni moltiplicate per i pesi, divisa per 15. Per tutte le dimensioni un valore più basso è migliore.

I numeri sono pochi e vanno letti come tali. Il carico complessivo è basso o moderato: su sette partecipanti la media è 26,7 per il carico semplice e 29,7 per

	Competenze informatiche e SQL		Età	
	poche o nessuna, senza SQL ($n = 6$)	buone, con SQL ($n = 1$)	fino a 25 anni ($n = 5$)	oltre 25 anni ($n = 2$)
(M) Richiesta mentale	26,7	45	27,0	35,0
(F) Richiesta fisica	19,2	40	22,0	22,5
(T) Richiesta temporale	32,5	50	34,0	37,5
(P) Prestazione	27,5	40	29,0	30,0
(S) Sforzo	32,5	35	32,0	35,0
(R) Frustrazione	10,0	20	10,0	15,0
Carico semplice	24,7	38,3	25,7	29,2
Carico pesato	28,2	38,7	29,3	30,5
Peso medio maggiore	mentale (3,7)	temporale (4)	mentale e prestazione (3,6)	mentale e temporale (3,0)

Tabella 6.3: Il NASA-TLX aggregato per profilo dei partecipanti, valori medi per gruppo. Competenze informatiche e conoscenza di SQL dividono il gruppo nello stesso modo, con il solo P2 da una parte; l'età lo divide diversamente, perché P7 ha più di 25 anni ma non conosce SQL.

quello pesato, e nessuno supera 40. Il risultato è migliore per chi non conosce SQL: il suo carico medio è 24,7 nella forma semplice e 28,2 in quella pesata, contro 38,3 e 38,7 di chi lo conosce (Tabella 6.3). È un risultato positivo, perché sono proprio queste le persone a cui l'editor si rivolge: per costruire un evento con la linea temporale non serve conoscere SQL, e chi non lo conosce non ne risulta svantaggiato. Il gruppo di chi conosce SQL, però, è il solo P2, quindi la differenza va presa come un'indicazione e non come una misura. Anche tra chi non conosce SQL i valori non sono uniformi: P3, P4, P5 e P7 riportano un carico semplice intorno a 20, tra 19,2 e 20,8, mentre P1 e P6 arrivano a 33,3 e 35,8, vicino a P2.

Tra le dimensioni, la frustrazione è la più bassa per quasi tutti, con una media di 11,4, e nessuno supera 20: anche chi ha trovato il compito impegnativo non si è sentito scoraggiato. Le valutazioni più alte vanno alla richiesta temporale (35,0 in media) e allo sforzo (32,9). La richiesta fisica divide i partecipanti: quattro la valutano 5, mentre P1, P2 e P6 le danno tra 40 e 50. I pesi dicono infine che cosa ha contato di più: in media la richiesta mentale (3,4), quella temporale e la prestazione (3,3 ciascuna), cioè capire come costruire l'evento, il ritmo e la riuscita; la richiesta fisica (1,9) e la frustrazione (1,1) pesano meno. Fanno eccezione P1, per cui pesano soprattutto la prestazione e la richiesta fisica, cioè cliccare e trascinare, e P6, che assegna il peso massimo alla richiesta mentale e nessuno a quella temporale. Il dato

più netto, infine, non è nei valori ma nell'attività che manca: il confronto con il testo ISEQL non è stato possibile, perché la notazione si è rivelata fuori portata per tutti i sette partecipanti, compreso l'unico che conosce SQL. Per chi non conosce ISEQL, l'editor grafico non è un'alternativa più comoda alla scrittura dell'espressione: è l'unica strada percorribile.

6.5.2 Valutazione euristica dell'interfaccia

Due osservazioni degli utenti dicono dove l'interfaccia non funziona, non perché. Per precisarle l'editor è stato esaminato con la **valutazione euristica** di Nielsen [20], un metodo di ispezione che non richiede utenti: si ripercorre il compito principale, qui creare un evento con la linea temporale a partire dalla pagina di analisi, e ogni problema incontrato viene ricondotto al principio di usabilità che viola e classificato per gravità, da 0 (non è un problema) a 4 (impedisce l'uso). I principi sono le dieci euristiche di Nielsen: (E1) visibilità dello stato del sistema, (E2) corrispondenza con il mondo reale, (E3) controllo e libertà dell'utente, (E4) coerenza e standard, (E5) prevenzione degli errori, (E6) riconoscimento anziché ricordo, (E7) flessibilità ed efficienza, (E8) design essenziale, (E9) aiuto a riconoscere e correggere gli errori, (E10) aiuto e documentazione. L'esame è stato condotto sull'applicazione in esecuzione, e la Tabella 6.4 ne riporta i risultati.

I due problemi segnalati dagli utenti sono i primi della tabella, e la valutazione ne spiega la causa comune: il sistema chiede di *ricordare* dove stanno le cose e quali gesti usare, invece di mostrarli. Gli altri problemi sono della stessa natura, e il terzo e il quarto aggravano il secondo: chi supera il riquadro vuoto e disegna la prima barra trova un elenco di predicati che al clic non risponde come si aspetta, e per dire "la stessa persona" deve passare per un concetto, l'insieme, che non ha chiesto.

La valutazione trova anche dei punti di forza, che vanno conservati. Lo stato del sistema è visibile dove conta: il distintivo tra due barre mostra subito la relazione che il disegno esprime, passando alla vista testuale si trovano l'espressione ISEQL e il SQL già aggiornati all'ultima modifica, e un errore nel testo è segnalato mentre si scrive. L'eliminazione di un evento salvato chiede conferma.

6.5.3 Cosa significa

Le difficoltà riguardano il modo di arrivare alle funzioni, non le funzioni: nessuno dei problemi trovati tocca il modello di evento, il compilatore o la rilevazione, e ciascuno ha una correzione circoscritta. Per questo sistema, però, l'interfaccia non è un dettaglio. L'utente a cui l'editor si rivolge è per costruzione non esperto,

	Problema	Eur.	G.	Correzione proposta
1	Per creare un evento si va dalla pagina di analisi alla configurazione, che si apre sui predicati; tra gli eventi il comando è un "+" di pochi pixel, senza etichetta, uguale ai controlli secondari.*	E6, E4	3	Un pulsante "Nuovo evento" con etichetta, già dalla pagina di analisi.
2	La linea temporale vuota non indica cosa fare: l'unico aiuto è una riga di testo grigio sopra il riquadro, e il gesto per creare un intervallo, trascinare, va conosciuto. L'alternativa è un secondo "+" senza etichetta, nel riquadro *Intervals*.*	E10, E6	3	Un invito nel riquadro vuoto ("trascina qui per il primo fatto") e un pulsante "Aggiungi un fatto" con etichetta.
3	Nella finestra dell'intervallo, cliccare un predicato non lo sceglie: propone di aprire la configurazione di quella relazione. Il predicato si inserisce solo trascinandolo nel riquadro o scrivendolo.	E4, E5	3	Il clic sceglie il predicato; la configurazione su un'icona a parte.
4	Una condizione ("la stessa persona") o una durata si aggiunge solo in un insieme, e solo dopo avervi assegnato gli intervalli uno per uno: tre punti dell'interfaccia per un'idea sola.	E7, E2	3	Condizioni aggiungibili anche senza insieme, per esempio dal distintivo tra due barre.
5	Il vocabolario è quello della notazione, in inglese: *Predicates Expression*, *cross-conditions*, *projection*, `arg1`, `M1`, `st`, i simboli δ, ε, ζ, η, ρ.	E2	3	Etichette in linguaggio comune; i parametri avanzati nascosti per impostazione predefinita.
6	La condizione di studio scelta in alto (A, B, C) filtra i predicati: per unire immagini e suoni bisogna sapere di dover scegliere C.	E5, E2	2	La condizione ricavata dai predicati scelti.
7	Allo zoom iniziale le barre sono corte e il nome del predicato è troncato (`ning(pers`), leggibile solo ingrandendo.	E6, E8	2	Uno zoom iniziale che adatti le barre al riquadro.
8	Non esiste un comando per annullare, e *Clear* cancella l'evento in costruzione senza chiedere conferma.	E3, E5	2	Annulla e ripeti; una conferma su *Clear*.
9	Un evento salvato si elimina solo dal menu del clic destro, che nulla segnala.	E6	1	Un'icona di eliminazione visibile su ogni evento.

Tabella 6.4: I problemi di usabilità dell'editor a linea temporale, con le euristiche di Nielsen violate (da E1 a E10, elencate nel testo) e la gravità (G.), da 0 a 4. L'asterisco indica i due problemi segnalati anche dagli utenti.

l'operatore che osserva le telecamere e non chi conosce la notazione, e se il sistema fosse proposto come prodotto sarebbe questa la persona che lo usa. I problemi di gravità 3 della Tabella 6.4 sono quindi gli interventi prioritari, e la Sezione 7.3 li riprende.

Queste verifiche hanno limiti da dichiarare. La prova con gli utenti ha coinvolto un gruppo piccolo, senza compiti prestabiliti, e non ha misurato né il tempo né la correttezza degli eventi costruiti; il NASA-TLX è stato compilato da sette partecipanti, sei dei quali con poche o nessuna competenza informatica e senza conoscenza di SQL, e i suoi valori valgono come indicazione, non come misura. La valutazione euristica è il giudizio di un solo valutatore, che per di più conosce il sistema, mentre il metodo ne raccomanda da tre a cinque indipendenti [20]. Le conclusioni sono quindi indicazioni per l'interfaccia, non una misura della sua usabilità. Una valutazione strutturata sul modello di [19], con compiti di complessità crescente, tempi di completamento e un questionario sul carico di lavoro percepito come il NASA-TLX, resta da fare.

Capitolo 7

Conclusioni

7.1 Cosa è stato fatto

Questa tesi è partita da due sistemi complementari. WATCHOUT rileva eventi complessi in video di sorveglianza con query SQL su intervalli prodotti da modelli visione-linguaggio e audio, ed è verificabile per costruzione, ma i suoi sedici eventi erano scritti a mano nel codice: aggiungerne uno significava modificare il backend e riavviarlo. L'ISEQL Event Model Builder permette di disegnare un evento su una linea temporale e ne ricava l'espressione ISEQL, ma non esegue nulla. Il lavoro è consistito nel farne un sistema solo.

Il risultato è che un evento si disegna, si salva e da quel momento viene rilevato accanto a quelli predefiniti, senza scrivere ISEQL né SQL, senza modificare il codice e senza riavviare il servizio. Il Capitolo 6 lo mostra su un evento che il sistema non prevedeva.

I contributi che hanno reso possibile questo percorso sono cinque.

Un ponte tra due modelli dei dati. Il Builder ragiona su argomenti in posizione, WATCHOUT conserva i partecipanti come insieme non ordinato. La posizione è ricavata dal tipo, usando come sistema di tipi le firme dei predicati che il sistema già conservava per altri scopi.

Un compilatore da modello a SQL. Traduce intervalli, operatori con soglie, condizioni e proiezione in una query eseguibile, con lo scoping per analisi scritto nella query stessa. Ogni operatore è reso dalla funzione che WATCHOUT già usava: c'è una definizione sola per operatore nell'intero sistema, e questo ha chiuso una duplicazione che aveva già prodotto una divergenza silenziosa nei risultati.

Un registro unico. Gli eventi definiti dall'utente e quelli predefiniti sono righe della stessa tabella, compilate dallo stesso percorso al momento della rilevazione. Le

query scritte a mano non esistono più nell'applicazione: un evento è un dato, non codice.

Un editor nel browser. La linea temporale vive nella stessa applicazione in cui si analizzano i video, e un evento con condizioni e proiezione si costruisce senza scrivere un simbolo della notazione. Due decisioni la governano: il disegno dichiara l'operatore e non la soglia, perché una soglia è una decisione e non una misura; e ciò che si salva è l'operatore scelto, non quello ricalcolabile dalle coordinate.

Una verifica per equivalenza. I sedici eventi esistono nel sistema come modelli, e le occorrenze che producono coincidono con quelle delle query originali sui database registrati. Il criterio non è "l'evento è stato rilevato" ma "sono state rilevate le stesse occorrenze", distinzione che ha fatto emergere almeno una divergenza reale.

Il lavoro si è svolto su un sistema in sviluppo attivo da parte del suo autore, con due integrazioni dei suoi commit e un editor concorrente scritto in parallelo. Le decisioni sulla semantica sono state poste come domande e non risolte unilateralmente, e quasi tutte hanno prodotto una definizione condivisa dove prima ce n'erano due.

7.2 Limiti

I limiti riconosciuti sono di tre tipi.

Di espressività. Il modello è una catena piatta e non un albero, quindi una sottoquery non può essere argomento di un operatore; non esiste l'aggregazione, se non attraverso attributi che il ponte precalcola; il ponte collassa la molteplicità dei partecipanti della stessa classe, con una convenzione deterministica ma arbitraria. Nessuno dei sedici eventi richiede ciò che manca, ma è un limite del linguaggio realizzato, non del dominio.

Di interfaccia. Una soglia impostata sulla linea temporale è un numero fissato nel modello: per renderla regolabile a ogni rilevazione bisogna darle un nome nella vista testuale (Sezione 6.1), un passaggio che richiede di conoscere la notazione. La prova con gli utenti (Sezione 6.5) ha aggiunto due limiti, che per chi non conosce la notazione pesano più degli altri: il comando che crea un evento è un piccolo pulsante difficile da trovare, e la costruzione degli intervalli non è guidata oltre la spiegazione iniziale. La valutazione euristica della stessa sezione ne ha trovati altri sette, dal clic su un predicato che non lo sceglie alle condizioni raggiungibili solo attraverso un insieme (Tabella 6.4). Infine l'editor, quando si cancella un intervallo, non rinumera ciò che vi fa riferimento (Sezione 5.6): condizioni, operatori e soglie

possono finire in silenzio sugli intervalli sbagliati, e un evento modificato in questo modo va ricontrollato nella vista testuale prima di salvarlo.

Di validazione. Il sistema è verificato sui percorsi che sono stati percorsi. La prima ora di uso reale attraverso l'interfaccia ha fatto emergere quattro difetti che nessun confronto di risultati poteva vedere, e quel numero misura l'esposizione, non la qualità: più uso dovrebbe far emergere altro. Soprattutto, ciò che l'intera tesi presuppone, che una persona non pratica della notazione riesca a costruire un evento corretto con questo editor, è stato finora osservato e non misurato: la prova della Sezione 6.5 dice che gli utenti trovano il sistema efficace ma poco pratico, non quanti eventi corretti costruiscono né in quanto tempo.

7.3 Sviluppi futuri

Cinque direzioni, in ordine di quanto sono vicine.

Verificare il modello nel servizio, non solo nell'editor. Il registro accetta la riga come arriva, e a garantire che il modello compili è l'interfaccia, che lo tiene compilato mentre lo si costruisce. Una richiesta inviata fuori dall'editor può quindi scrivere un modello che nessuno potrà eseguire, e un modello che nomina un predicato mai prodotto dalla percezione compila e non trova nulla, il che è indistinguibile da un evento che non si è verificato. Sono due controlli da spostare nel servizio: compilare prima di scrivere, e confrontare i predicati con il vocabolario, come il compilatore fa già quando lavora per l'editor.

Provare un evento prima di salvarlo. La cosa più utile che manca: eseguire il modello in costruzione su un'analisi esistente e mostrare quante occorrenze trova, prima del salvataggio. Chi disegna un evento non sa se la soglia che ha scelto sia troppo stretta finché non lo salva e lo lancia; una anteprima dei risultati chiuderebbe il ciclo tra il disegno e i dati. Il meccanismo per eseguire una query esiste già, manca solo di esporlo per un modello non ancora registrato.

Analisi di sensibilità. Le soglie si regolano a ogni rilevazione, ma nulla le fa variare sistematicamente. Un'esecuzione che ripeta un evento per un intervallo di valori sulle trenta scene produrrebbe la curva di precisione e richiamo in funzione della soglia, che è l'informazione che serve per sceglierla. Anche qui la macchina c'è: serve il ciclo esterno.

Un'interfaccia per chi non conosce la notazione, e uno studio che la misuri. La prova con gli utenti e la valutazione euristica (Sezione 6.5) hanno indicato dove intervenire, con una correzione per ciascun problema (Tabella 6.4): prima di tutto un comando ben visibile per creare un evento, una costruzione degli

intervalli guidata passo per passo invece che affidata a gesti da conoscere, un clic che scelga il predicato e condizioni che non richiedano un insieme. Va corretta anche la cancellazione degli intervalli, facendola passare per le regole di `model_edit.py`, che il backend offre già. Il passo successivo è misurare: un gruppo di persone che non conoscono ISEQL, eventi di complessità crescente da esprimere a partire da una descrizione in italiano, il tempo e la correttezza di ciò che costruiscono e il carico di lavoro percepito, confrontati con l'editor testuale, come nella valutazione di [19]. È l'unico modo di sapere se l'ipotesi da cui la tesi parte regge, ed è la condizione per proporre il sistema a utenti reali.

Un esecutore alternativo per gli archivi. Le query generate usano join su disuguaglianze, il cui costo cresce con il quadrato del numero di intervalli. Su un'analisi di dieci secondi è irrilevante; su un archivio di mesi non lo sarebbe. Gli algoritmi a scansione del piano [3] valutano gli stessi operatori in tempo quasi lineare, e il repository contiene già un esecutore nativo di questo tipo. Collegarlo significherebbe generare, dallo stesso modello, non una query SQL ma un piano per quell'esecutore: il modello resterebbe lo stesso, cambierebbe solo il bersaglio della compilazione. È la prova che il modello di evento, e non il SQL, è la cosa che questa tesi ha effettivamente costruito.

Bibliografia

[1] Sven Helmer and Fabio Persia. High-level surveillance event detection using an interval-based query language. In *Proceedings of the 2016 IEEE Tenth International Conference on Semantic Computing (ICSC)*, pages 350–353. IEEE, 2016. doi: 10.1109/ICSC.2016.19.

[2] James F. Allen. Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11):832–843, 1983. doi: 10.1145/182.358434.

[3] Danila Piatov, Sven Helmer, Anton Dignös, and Fabio Persia. Cache-efficient sweeping-based interval joins for extended allen relation predicates. *The VLDB Journal*, 30(3):379–402, 2021. doi: 10.1007/s00778-020-00650-5.

[4] Sangmin Oh, Anthony Hoogs, Amitha Perera, Naresh Cuntoor, Chia-Chih Chen, Jong Taek Lee, Saurajit Mukherjee, J. K. Aggarwal, Hyungtae Lee, Larry Davis, Eran Swears, Xinyu Wang, Qiang Ji, Kishore Reddy, Mubarak Shah, Carl Vondrick, Hamed Pirsiavash, Deva Ramanan, Jenny Yuen, Philip Torr, Bi Song, Tomasz Matuszewski, and Soommin Joo. A large-scale benchmark dataset for event recognition in surveillance video. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011. doi: 10.1109/CVPR.2011.5995586.

[5] Joseph Redmon, Santosh Divvala, Ross Girshick, and Ali Farhadi. You only look once: Unified, real-time object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 779–788, 2016. doi: 10.1109/CVPR.2016.91.

[6] Qiuqiang Kong, Yin Cao, Turab Iqbal, Yuxuan Wang, Wenwu Wang, and Mark D. Plumbley. PANNs: Large-scale pretrained audio neural networks for audio pattern recognition. *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, 28:2880–2894, 2020. doi: 10.1109/TASLP.2020.3030497.

[7] Fabio Persia, Fabio Bettini, and Sven Helmer. Labeling the frames of a video stream with interval events. In *Proceedings of the 2017 IEEE 11th International Conference on Semantic Computing (ICSC)*, pages 204–211. IEEE, 2017. doi: 10.1109/ICSC.2017.55.

[8] Viviana Crescitelli, Fabio Persia, Daniele Cipriani, and Francesco Pea. VIS MODE: Complex event detection from vlm-based video observations. In *European Conference on Computer Vision (ECCV), Demo Track*, 2026.

[9] Sven Helmer and Fabio Persia. ISEQL, an interval-based surveillance event query language. *International Journal of Multimedia Data Engineering and Management (IJMDEM)*, 7(4):1–21, 2016.

[10] Fabio Persia, Fabio Bettini, and Sven Helmer. An interactive framework for video surveillance event detection and modeling. In *Proceedings of the 26th ACM International Conference on Information and Knowledge Management (CIKM '17)*, pages 2515–2518. ACM, 2017. doi: 10.1145/3132847.3133164.

[11] Fabio Persia, Anton Dignös, Sven Helmer, Johann Gamper, and Daniela D'Auria. Modeling and detecting high-level events in healthcare applications exploiting ISEQL+. *Soft Computing*, 30(5):3667–3690, 2026. doi: 10.1007/s00500-025-10716-7.

[12] Rui Z. Barbosa, Hugo S. Oliveira, and João Manuel R. S. Tavares. A survey on multi-modal and weakly supervised approaches for robust anomaly detection in video data. *Information Fusion*, 126:103388, 2026. doi: 10.1016/j.inffus.2025.103388.

[13] Yunlong Tang, Jing Bi, Siting Xu, Luchuan Song, Susan Liang, Teng Wang, Daoan Zhang, Jie An, Jingyang Lin, Rongyi Zhu, Ali Vosoughi, Chao Huang, Zeliang Zhang, Pinxin Liu, Mingqian Feng, Feng Zheng, Jianguo Zhang, Ping Luo, Jiebo Luo, and Chenliang Xu. Video understanding with large language models: A survey. *IEEE Transactions on Circuits and Systems for Video Technology*, 36(2):1355–1376, 2026. doi: 10.1109/TCSVT.2025.3566695.

[14] Pietro Bellodi, Pietro Casavecchia, Alberto Paparella, Guido Sciavicco, and Ionel Eduard Stan. Assessing the (in)ability of LLMs to reason in interval temporal logic. In *Proceedings of the 32nd International Symposium on Temporal Representation and Reasoning (TIME)*, pages 4:1–4:15, 2025. doi: 10.4230/LIPIcs.TIME.2025.4.

[15] Yunfei Chu, Jin Xu, Qian Yang, Haojie Wei, Xipin Wei, Zhifang Guo, Yichong Leng, Yuanjun Lv, Jinzheng He, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen2-audio technical report. *arXiv preprint arXiv:2407.10759*, 2024.

[16] Ghiffary Rifqialdi. WATCHOUT ISEQL: Large audio-language models and object re-identification for forensic multimodal surveillance event detection. Tesi di laurea magistrale, Università degli Studi dell'Aquila e Universität Hamburg, 2026. In preparazione.

[17] ByteDance Seed. Seedance 2.0: Advancing video generation for world complexity. *arXiv preprint arXiv:2604.14148*, 2026.

[18] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 11975–11986, 2023.

[19] Massimiliano Albanese, Antonio d'Acierno, Vincenzo Moscato, Fabio Persia, and Antonio Picariello. A multimedia recommender system. *ACM Transactions on Internet Technology*, 13(1):3:1–3:32, 2013. doi: 10.1145/2532640.

[20] Jakob Nielsen. Heuristic evaluation. In Jakob Nielsen and Robert L. Mack, editors, *Usability Inspection Methods*, pages 25–62. John Wiley & Sons, New York, 1994.